


# RFC -- NomadPlan

**Request for Comments**

**Projeto: Central de Roteiros Personalizados**

**Engenharia de Software – Católica SC**

**Sandro Marcos Machiniski - novembro de 2026**

**Link do GitHub: https://github.com/SandroMachiniski-SE/NomadPlan.git**

# Sumário

**Identificação** ... **3**
**1 Introdução** ... **3**
**2 O Problema** ... **3**
**3 Objetivos** ... **4**
3.1 Contexto do Problema ... 4
3.2 Origem da Demanda e Evidências ... 7
3.3 Pesquisa com Usuários ... 7
3.4 Análise de Soluções Existentes (Benchmark) ... 11
**4 Público-Alvo** ... **14**
**5 Objetivos do Projeto** ... **14**
5.1 Objetivo Geral ... 14
5.2 Objetivos Específicos ... 15
5.3 Métricas de Sucesso (KPIs) ... 15
**6 Engenharia de Requisitos** ... **18**
6.1 Personas ... 18
6.2 Casos de Uso Principais ... 19
6.3 Requisitos Funcionais (RF) ... 22
6.4 Requisitos Não Funcionais (RNF) ... 24
6.5 Regras de Negócio (RB) ... 27
6.6 Fora do Escopo ... 29
**7 Fluxos e Comportamento do Sistema** ... **31**
7.1 Fluxo Principal do Usuário ... 31
7.2 Fluxos Alternativos ... 32
**8 Mockups e Experiência do Usuário (UX)** ... **37**
8.1 Fluxo de Navegação ... 38
8.2 Wireframes ou Mockups das Telas ... 40
8.3 Fluxo de Interação do Usuário ... 51
8.4 Feedback Inicial de Usuários (Opcional) ... 52
**9 Arquitetura do Sistema** ... **53**
9.1 Diagrama C4 ... 53
9.2 Modelo de Dados ... 59
9.3 Principais Componentes ... 61
9.4 Stack Tecnológica ... 62
9.5 Origem dos Dados ... 62
**10 Segurança e Privacidade** ... **63**
10.1 Privacidade e LGPD ... 64
**11 Planejamento do Projeto** ... **65**
**12 Referências** ... **66**
**13 Apêndices** ... **67**
**14 Parecer do Comitê de Avaliação** ... **69**

## Identificação

- Projeto: NomadPlan
- Tipo: Plataforma Social de Turismo Inteligente
- Disciplina: Engenharia de Software
- Data: Novembro de 2026
- Autor: Sandro Marcos Machiniski
- Versão: 2.0

# 1 Introdução

Contexto e motivação No contexto atual, o turismo é uma das principais atividades econômicas e culturais em nível global, e a digitalização dos serviços tem transformado a forma como viajantes planejam e vivenciam suas experiências. Apesar da vasta oferta de informações dispersas em múltiplas plataformas, ainda existe uma lacuna na centralização e personalização de dados turísticos locais — especialmente no que tange inventários urbanos, roteiros personalizados e integração entre recomendações e perfis de usuários. Este projeto propõe o desenvolvimento de uma aplicação web/mobile que centraliza inventários turísticos das cidades, facilitando a descoberta, pesquisa e planejamento de viagens, além de promover interação social entre usuários com gostos semelhantes.

# 2 O Problema

A oferta de informações turísticas encontra-se fragmentada em múltiplas fontes (sites institucionais, blogs, redes sociais, plataformas de reservas e guias locais), o que dificulta o planejamento eficiente de viagens por parte dos usuários e reduz a visibilidade de atrativos e serviços de menor porte. Essa dispersão gera inconsistências nos dados (horários, preços, condições de

acessibilidade), aumento do tempo despendido na pesquisa e perda de oportunidades econômicas para negócios locais. Além disso, as recomendações disponíveis em muitas plataformas não consideram de forma adequada as preferências individuais e o contexto social do viajante, resultando em roteiros genéricos que nem sempre correspondem às expectativas ou necessidades (por exemplo, acessibilidade, orçamento, interesses culturais).

# 3 Objetivos

Desenvolver uma plataforma para catalogação e pesquisa de pontos e áreas turísticas, capaz de sugerir roteiros personalizados com base no perfil e preferências dos usuários, funcionando também como uma rede social voltada ao turismo. A plataforma visa unificar informações locais (atrativos, horários, preços, acessibilidade, categorias) e fornecer recomendações contextuais e sociais que auxiliem tanto viajantes quanto gestores locais.

## 3.1 Contexto do Problema

**- Quem sofre com esse problema**

Viajantes (individuais e em grupo), pequenos negócios locais (guias, restaurantes, pousadas), gestores públicos e comunidades locais, que perdem visibilidade, clientes e base de dados confiável para planejamento.

**- Em que contexto ele ocorre**

Em um ambiente online fragmentado, com informações dispersas entre sites, redes sociais, guias e plataformas de reserva, além da alta variabilidade e atualização frequente de dados (horários, preços, acessibilidade).

**- Como esse problema é resolvido atualmente**

Por agregadores e grandes plataformas de viagens, serviços de mapas (Google Maps, OpenStreetMap), redes sociais e portais institucionais ou guias locais, além de agências e guias tradicionais.

**- Quais são as limitações das soluções atuais**

Fragmentação e inconsistência dos dados; informações desatualizadas; pouca personalização das recomendações; baixa representatividade de pequenos atores locais; custo e barreiras técnicas para manutenção de inventários; limitações na integração entre dados geoespaciais e perfis de usuário.

**- Exemplos reais de crescimento e demanda**






**O que fazer em Joinville** é uma dúvida comum para quem visita o norte de Santa Catarina. Conhecida como a "Cidade das Flores" e famosa por sediar o maior festival de dança do mundo, a cidade oferece uma combinação de cultura, natureza, boa comida e tradição germânica que conquistam turistas o ano inteiro.

## O que fazer em Joinville: Roteiros sugeridos

### Roteiro cultural (1 dia)

- **Museu Nacional de Imigração e Colonização** (1h a 1h30 de visita).
- Caminhada até a **Rua das Palmeiras**, localizada a poucos metros do museu.
- **Museu de Arte de Joinville (MAJ)** (aprox. 1h de visita), a menos de 10 minutos a pé.
- Encerramento com jantar típico no restaurante Biergarten Joinville.

### Roteiro natureza e lazer (1 dia)

- Subida ao Mirante do Boa Vista
- Passeio pela Estrada Bonita (almoço em restaurante típico)
- Bate-volta até São Francisco do Sul para passeio de barco na Baía da Babitonga

### Roteiro completo (2 dias)

- Dia 1: museus, Rua das Palmeiras e Festival de Dança (se for julho)
- Dia 2: Mirante do Boa Vista + Estrada Bonita + jantar típico
- Opcional: passeio de barco na Baía da Babitonga, saindo de São Francisco do Sul

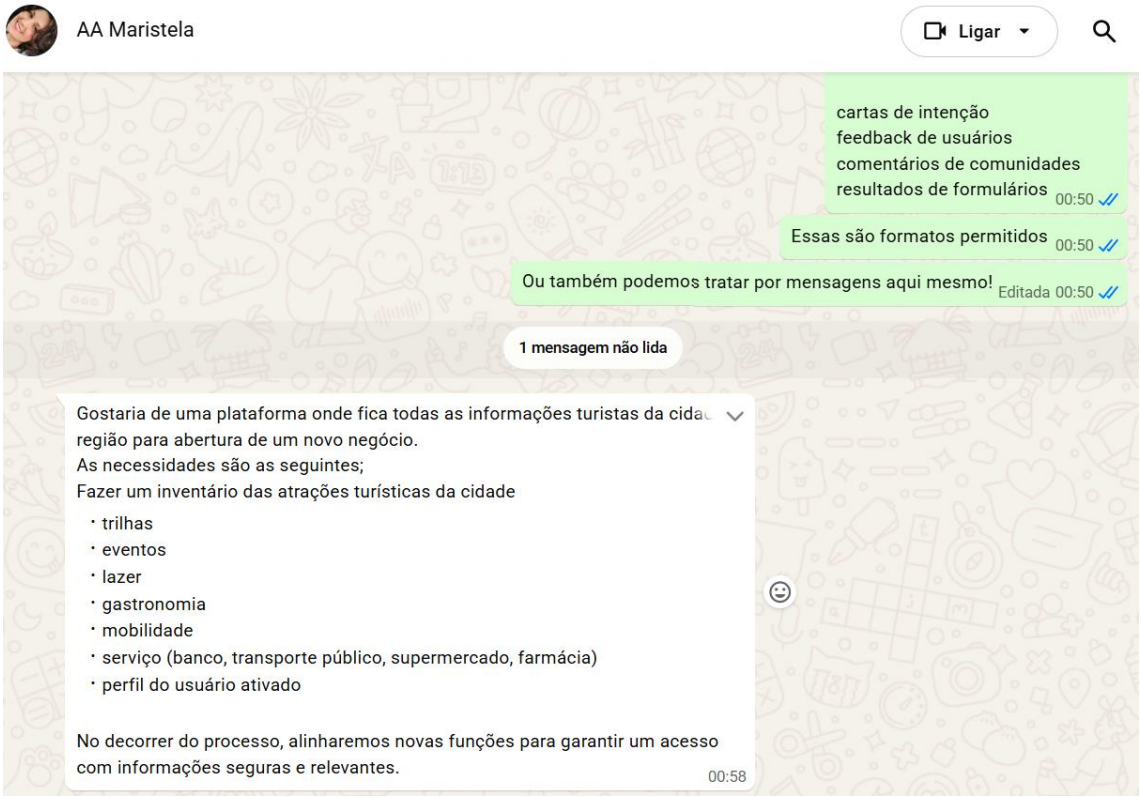
### Roteiro para famílias (1 dia)

- Parque Zoobotânico, com trilhas leves e observação de animais
- Almoço em restaurante rural na Estrada Bonita
- Passeio tranquilo pela Rua das Palmeiras e visita ao Museu Nacional de Imigração

## Principais pontos turísticos

- **Museu Nacional de Imigração e Colonização**
  Rua Rio Branco, 229 – Centro
  Terça a domingo, das 10h às 16h
  R$ 5,00 (meia-entrada para estudantes e idosos)
  Tempo de visita: 1h a 1h30
  **Confira mais informações no site oficial:** www.joinville.sc.gov.br
- **Rua das Palmeiras**
  Uma das ruas mais fotogênicas do Brasil, ladeada por palmeiras imperiais e casarões históricos.
- **Mirante do Boa Vista**
  Vista panorâmica da cidade após subida de 200 degraus.

## 3.2 Origem da Demanda e Evidências




**Demanda Externa**

**Projeto solicitado por:**

**Maristela Alexandre**

## 3.3 Pesquisa com Usuários

# Pesquisa de Campo para Projeto de Sistema Turístico

O objetivo desta pesquisa é descobrir lacunas e deficiências na gestão e organização do turismo regional

Nome *

Texto de resposta curta

Qual é a sua relação com o turismo na região? *

- ◯ Empresário do setor
- ◯ visitante ocasional
- ◯ visitante frequente

Qual a maior dificuldade que você encontra ao buscar informações sobre atrações e serviços turísticos na região? *

Texto de resposta longa

Você costuma encontrar informações desatualizadas ou conflitantes sobre horários, preços ou disponibilidade?

- ◯ Nunca
- ◯ Raramente
- ◯ Às vezes
- ◯ Frequentemente
- ◯ Sempre

O que mais você sugeriria para que a informação turística da região fosse mais útil e acessível para moradores e visitantes? *

Texto de resposta longa

**- entrevistas**

**- questionários**

9 respostas

Link para as Planilhas

Resumo | Pergunta | Individual

Nome

9 respostas

Jair Nunes

Gabrielli Lorenzi

Fabiane Machiniski Lorenzi

Cleverson Fermiano dos Santos

Maristela Alexandre

Guaracy Vieira

Valentina

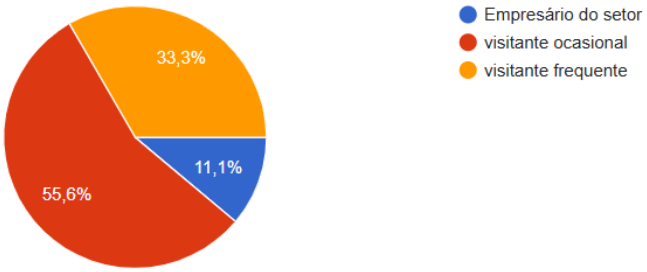
Geyson Amaral

Daniela Stefani Schmitz Machiniski

Qual é a sua relação com o turismo na região?

Copiar gráfico

9 respostas




Qual a maior dificuldade que você encontra ao buscar informações sobre atrações e serviços turísticos na região?

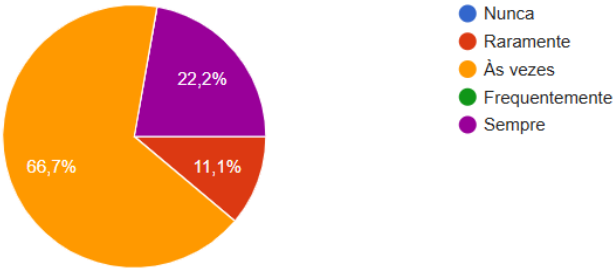
9 respostas

- Informações oficiais. Essas são mais raras. Não há dificuldade em obter informações com amigos ou outros grupos de trilha.
- Falta de informações completas sobre os passeios
- Aqui está sendo bem divulgado
- Falta de conhecimento em saber que os lugares existem.
- Falta de divulgação e aplicativo.
- Atendimento nas áreas de turismo, nos receptivos.
- Falta de canal de comunicação e falta de divulgação
- Não tem ou não conheço uma lista de atrações com horários, preços ,localização, serviços,
- Informações claras sobre valores

Você costuma encontrar informações desatualizadas ou conflitantes sobre horários, preços ou disponibilidade?

Copiar gráfico

9 respostas



O que mais você sugeriria para que a informação turística da região fosse mais útil e acessível para moradores e visitantes?

9 respostas

Penso que nos dias de hoje às ferramentas mais acessíveis estão nos meios digitais, então a criação de um aplicativo contendo informações sobre as atrações turísticas da região, seria ótima.

Informações completas e atualizadas frequentemente, com passeios disponíveis (dia e horário) quanto tempo de duração, se possuiu guia ou não.

Mais cartazes instrutivos

Divulgação em propagandas que chegue em outras regiões.

Algo atualizado com opções para lazer, gastronomia, trilhas, passeios e eventos.

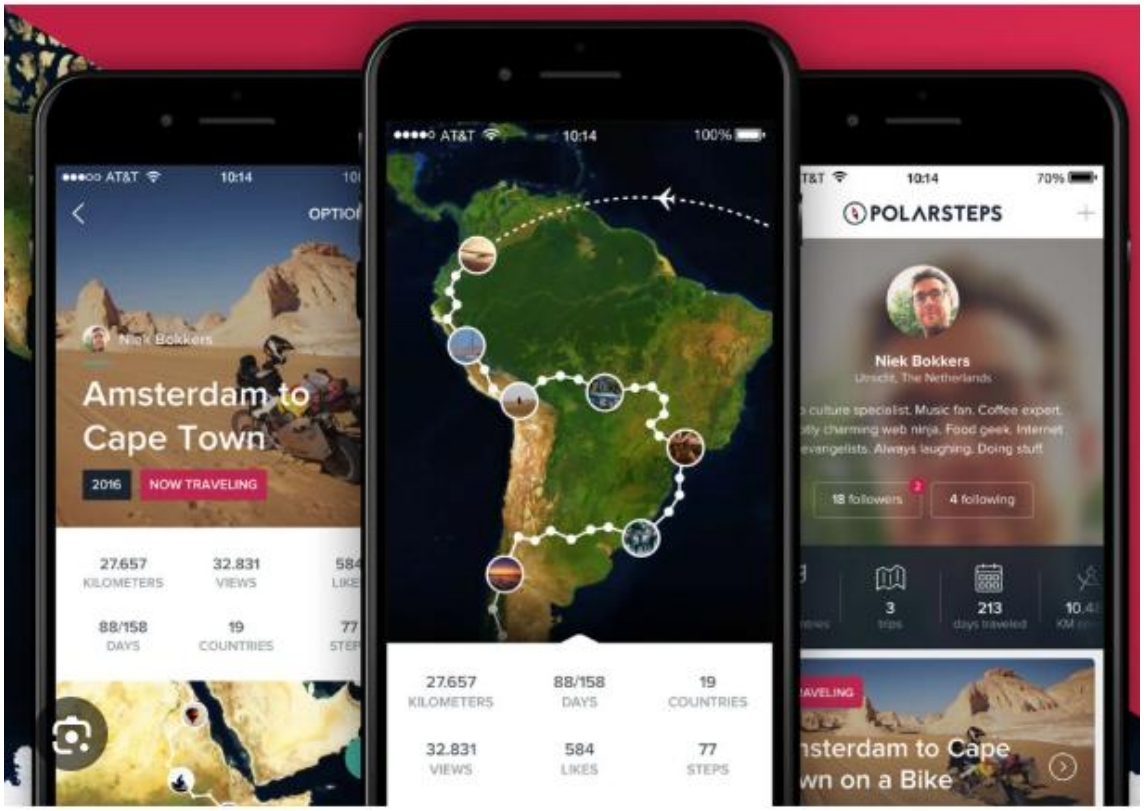
Retorna-se os atendimento ao turista nas estradas da Cidades não tem ninguém para o receptivo divulgando os atrativos turísticos.

Mais divulgação melhor sinalização e segurança

Talvez um aplicativo,algum canal que tenha essas informações e que também possa tirar dúvidas,como

## 3.4 Análise de Soluções Existentes (Benchmark)

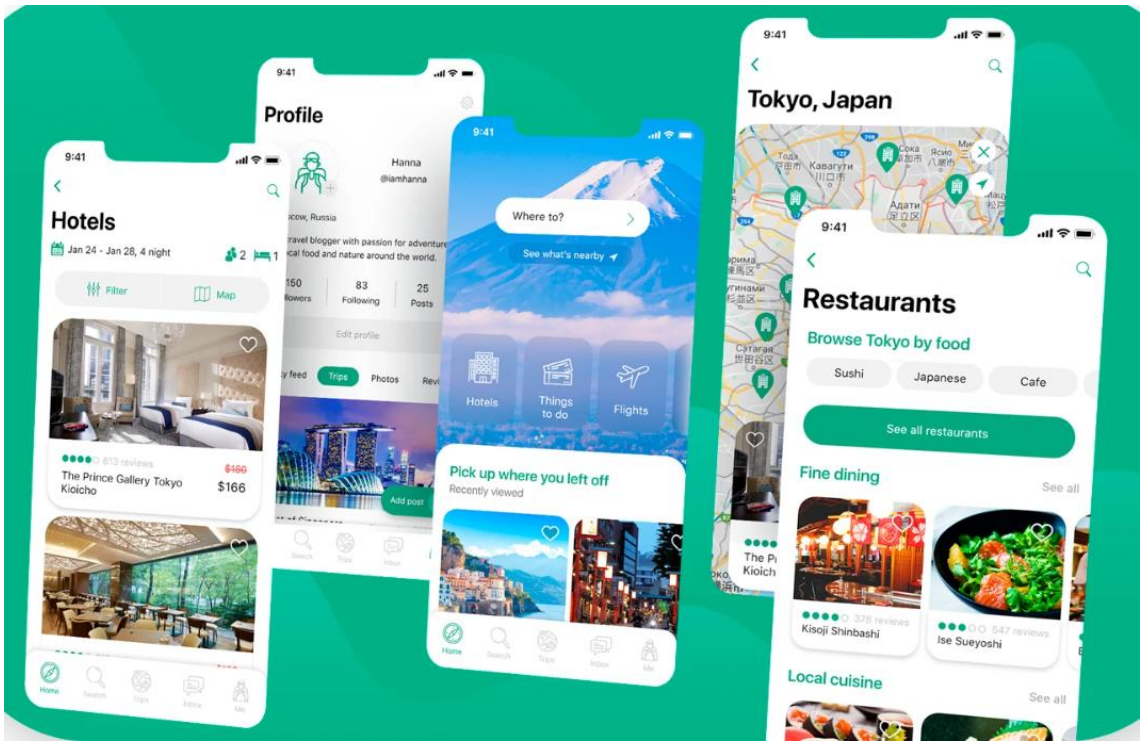
### Aplicativo polarsteps



app-macamp-geral




**Tripadvisor**




**Diferencial do Projeto**

**Por que criar algo novo**

- **O turismo é uma área com grande volume de informações dispersas (sites de prefeituras, blogs, redes sociais, aplicativos de viagem).**
- **Criar uma plataforma única e integrada facilita o acesso e melhora a experiência do usuário.**
- **Além disso, o projeto serve como vitrine da sua capacidade de aplicar conceitos de engenharia de software em um problema real, com arquitetura, banco de dados, integração de APIs e interface moderna.**

**Qual lacuna não foi resolvida pelas soluções existentes**

- **Fragmentação das informações: hoje, os turistas precisam consultar múltiplas fontes (Google Maps, TripAdvisor, sites oficiais, blogs).**
- **Falta de personalização: a maioria dos aplicativos sugere roteiros genéricos, sem considerar o perfil individual do usuário.**
- **Pouca interação social: plataformas atuais não funcionam como redes sociais de turismo, onde viajantes podem compartilhar experiências e roteiros.**
- **Gestão local limitada: cidades menores não têm ferramentas acessíveis para divulgar seus inventários turísticos de forma centralizada.**

**Qual nicho específico será atendido**

- **Turistas que buscam roteiros personalizados: pessoas que querem experiências adaptadas ao seu gosto (gastronomia, aventura, cultura, natureza).**
- **Cidades de médio e pequeno porte: que muitas vezes ficam fora dos grandes aplicativos de turismo e precisam de uma plataforma para divulgar seus atrativos.**
- **Comunidade de viajantes: usuários que desejam compartilhar avaliações, dicas e roteiros, criando uma rede social voltada exclusivamente para turismo.**

# 4 Público-Alvo

- Visitantes / Turistas
- Moradores locais
- Pequenos negócios turísticos
- Gestores públicos e órgãos de turismo municipal / regional
- Proprietários / responsáveis por atrativos
- Guias e serviços turísticos (guias independentes, operadores de passeios)
- Pesquisadores e estudantes (universitários, institutos de pesquisa)
- Investidores e empreendedores

**Observações**

- **Perfil de usuário:** Visitante / Turista — Usuário final que busca informações e roteiros
- **Contexto de uso:** Planejamento de viagens, busca por atrações e roteiros, salvamento de lugares, check-ins e avaliações após visitas.
- **Nível de conhecimento técnico esperado:** Baixo a moderado — usam apps/sitios comuns (mapas, buscadores), não requerem passos técnicos complexos.

# 5 Objetivos do Projeto

### 5.1 Objetivo Geral

Desenvolver uma plataforma integrada para gestão e descoberta de recursos turísticos que centralize inventários por cidade, viabilize a pesquisa e filtragem de atrações, e ofereça roteiros personalizados com base no perfil e preferências dos usuários, promovendo maior visibilidade para negócios locais e fornecendo dados úteis para gestores públicos.

Essa transformação busca unir dados geoespaciais, conteúdos verificados e mecanismos sociais/recomendação para tornar o planejamento de viagens mais eficiente, inclusivo e orientado às necessidades reais dos visitantes e da comunidade local.

### 5.2 Objetivos Específicos

- Automatizar o processo manual de cadastro e atualização de inventários turísticos.
- Centralizar e padronizar informações georreferenciadas sobre atrativos e serviços locais.
- Permitir a análise de dados e geração de relatórios para apoiar decisões de gestores e empreendedores.
- Criar um sistema de recomendação que sugira roteiros personalizados com base no perfil e preferências do usuário.
- Promover maior visibilidade e inclusão de pequenos negócios e atrativos menos conhecidos.

### 5.3 Métricas de Sucesso (KPIs)

- Latência média de busca < 300 ms
- Tempo médio de geração de roteiro < 5 s
- Acurácia/precisão do motor de recomendação ≥ 75% (ou taxa de satisfação do usuário ≥ 70%)
- Suporte a 100 usuários simultâneos na demonstração/ambiente de teste
- Redução do tempo de cadastro/atualização manual em ≥ 40%

KPIs detalhados por categoria

1) Desempenho técnico

- Latência de buscas (relevância: experiência): latência média por requisição de busca < 300 ms (pico < 1 s).
- Tempo de geração de roteiro (recomendações + roteamento): média < 5 segundos para gerar um roteiro em ambiente de teste.
- Disponibilidade: uptime ≥ 99% durante a banca/demonstração.
- Capacidade concorrente: suportar 100 usuários simultâneos com degradação de performance controlada (ex.: latência aumente no máximo 2×).

2) Qualidade e governança de dados

- Cobertura mínima de dados: pelo menos 100 pontos turísticos registrados e com metadados completos (campos obrigatórios preenchidos).
- Taxa de completude dos cadastros: ≥ 85% dos registros com principais campos preenchidos (nome, coordenadas, horário, preço/indicação e imagens).
- Taxa de dados verificados: ≥ 30% dos negócios/atrativos com selo de verificação (documento/validação).
- Taxa de conflitos ou duplicidades detectadas e resolvidas automaticamente: ≤ 5%.

3) Adoção e uso

- Usuários ativos (durante testes/piloto): ≥ 50 usuários ativos únicos no período de avaliação.
- Retenção de usuários no piloto: ≥ 40% dos usuários retornam após uma semana.
- Ações por usuário (média): pelo menos 3 ações relevantes por usuário (buscas, salvamentos, avaliações, criação de roteiros).
- Engajamento em conteúdo social: ≥ 20% dos usuários contribuem com avaliação/foto durante o piloto.

4) Experiência do usuário (UX)

- Satisfação geral (NPS ou escala 1–5): média ≥ 4 de 5, ou NPS > 20 em grupo de teste.
- Taxa de sucesso em tarefas (teste de usabilidade): ≥ 80% das tarefas críticas completadas sem ajuda.
- Tempo médio para cadastrar um novo ponto (fluxo guiado): redução de 40% comparado ao processo manual (ex.: de 15 min para ≤ 9 min).

5) Recomendação e qualidade do roteiro

- Acurácia/qualidade das recomendações: métricas podem variar — sugiro medir via:
- Precisão@K (por exemplo, Precision@5) ≥ 0.6 em avaliações automáticas; e/ou
- Satisfação do usuário com roteiro gerado ≥ 70% em pesquisa pós-uso.
- Cobertura de recomendações: ao menos 70% dos perfis de teste recebem roteiros com pelo menos 3 pontos relevantes.
- Taxa de aceitação do roteiro (usuários que salvam/seguem o roteiro gerado): ≥ 30% durante o piloto.

6) Segurança e governança

- Autenticação e autorização: 100% das rotas sensíveis protegidas por controle de acesso.
- Logs/auditoria: 100% das operações CRUD críticas registradas com metadados (usuário, timestamp, alteração).
- Conformidade de privacidade: mecanismos de consentimento implementados e documentados.
- Como medir na prática (instrumentação)
- Use métricas de aplicação: Prometheus + Grafana (ou ferramentas equivalentes) para latência, throughput e disponibilidade.
- Logs e auditoria: ELK stack ou serviço similar para rastrear alterações, auditorias e erros.
- Analytics de uso: Google Analytics / Mixpanel / Matomo para acompanhar usuários ativos, retenção e eventos (salvar, compartilhar, avaliar).

- Testes de carga: Gatling, JMeter ou k6 para validar capacidade concorrente.
- Avaliação da recomendação: testes A/B e questionários pós-uso (satisfação, utilidade) com grupo piloto de usuários.

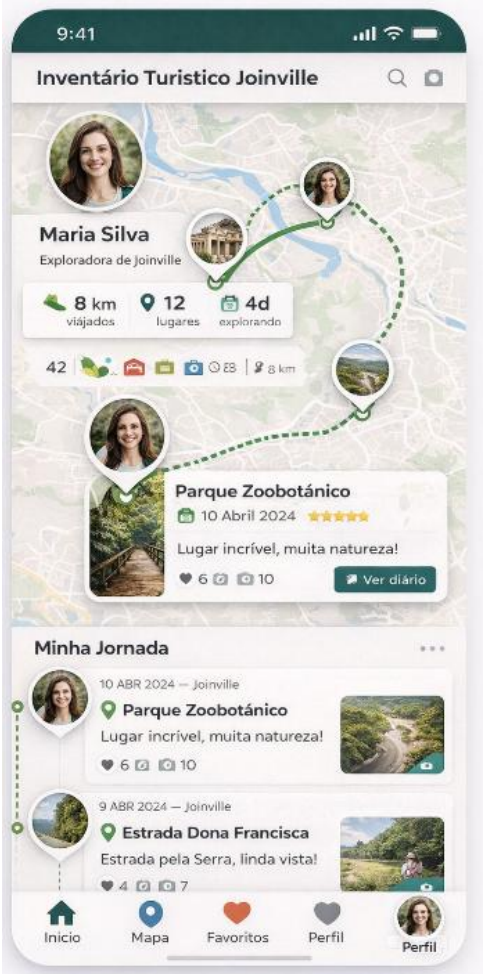
# 6 Engenharia de Requisitos

## 6.1 Personas

Persona 1 — Maria, a Turista Planejadora

- Contexto: Maria tem 32 anos, mora em outra cidade e costuma viajar fins de semana e feriados; pesquisa tudo online antes de ir.
- Objetivos: descobrir atrações relevantes na cidade de destino; montar um roteiro para 1–2 dias que encaixe em seu orçamento e preferências (cultura e gastronomia); acessar informações atualizadas sobre horários e reservas.
- Principais dificuldades: informações dispersas e desatualizadas, dificuldade em encontrar opções acessíveis por região, perda de tempo montando roteiros manualmente.



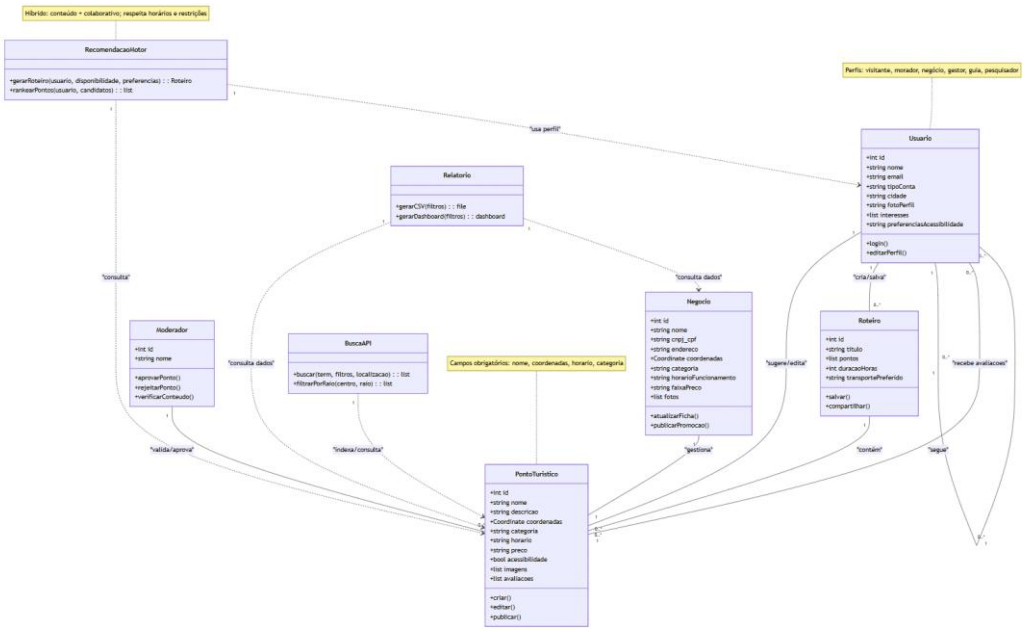




### 6.2 Casos de Uso Principais

- **Criar conta**
    - Descrição: usuário (visitante, morador, negócio, gestor ou guia) realiza cadastro, escolhe tipo de conta e completa perfil inicial.
    - Entradas: e-mail, senha, tipo de conta, dados básicos (nome/razão social, cidade).
    - Saídas: conta criada, onboarding direcionado.
- **Autenticar / Recuperar senha**
    - Descrição: login via e-mail/senha e recuperação de credenciais por e-mail.
    - Entradas: credenciais do usuário.
    - Saídas: token/ sessão ativa; acesso às funcionalidades conforme permissão.
- **Registrar / Cadastrar ponto turístico (CRUD)**

    - Descrição: criação e edição de registros de atrativos e serviços (nome, endereço, coordenadas, categoria, horários, preço, acessibilidade, imagens, contato).
    - Entradas: formulário de cadastro, uploads de imagens, coordenadas GPS.
    - Saídas: ponto salvo em rascunho ou publicado (dependendo do fluxo de verificação); registro em histórico de versões.
- **Validar / Verificar ponto (fluxo de moderação)**
    - Descrição: moderador ou proprietário verifica a autenticidade das informações; selo “verificado” pode ser aplicado.
    - Entradas: solicitação de verificação, documentos/provas, checagens automáticas.
    - Saídas: ponto marcado como verificado, notificação ao cadastrado.
- **Sugerir/Editar dado via crowdsourcing**
    - Descrição: usuários enviam atualizações (horários, fotos, avisos de fechamento); sistema processa e aplica conforme regras de confiança.
    - Entradas: proposta de edição, evidências (foto, link).
    - Saídas: alteração aplicada imediatamente ou submetida à revisão; histórico de mudanças.
- **Pesquisar e filtrar pontos / exploração por mapa**
    - Descrição: busca por cidade, nome, categoria; filtros por acessibilidade, preço, distância, avaliações; visualização em lista e mapa.
    - Entradas: termo de busca, filtros, localização atual.
    - Saídas: resultados ordenados por relevância e proximidade; links para detalhes.
- **Visualizar detalhe do ponto**
    - Descrição: exibição de ficha completa com fotos, horários, avaliações, rota e botão para adicionar ao roteiro ou salvar.
    - Entradas: seleção do ponto.
    - Saídas: página de detalhe, possibilidade de interação (avaliar, comentar, salvar).

- **Gerar roteiro personalizado (motor de recomendação)**
    - Descrição: usuário define preferências e disponibilidade; sistema sugere roteiro otimizado por tempo, categorias e localização.
    - Entradas: perfil do usuário (interesses), tempo disponível, ponto de partida, preferências de transporte.
    - Saídas: roteiro com sequência de visitas, tempos estimados e mapa; opção de salvar/compartilhar.
- **Criar / Editar / Compartilhar roteiro (social)**
    - Descrição: usuário personaliza roteiro, adiciona notas e compartilha com outros usuários ou pública.
    - Entradas: seleção de pontos, ordem desejada, notas e configurações de privacidade.
    - Saídas: roteiro salvo, link público/privado compartilhável.
- **Avaliar e comentar ponto / seguir usuário**
    - Descrição: interação social que alimenta reputação e recomendações (avaliação por estrelas, comentários, fotos).
    - Entradas: avaliação, texto, imagens.
    - Saídas: avaliação publicada, impacto no score do ponto e na reputação do autor.
- **Exportar / Relatórios para gestores**
    - Descrição: geração de relatórios e exportação de dados (CSV/GeoJSON) para órgãos e empreendedores.
    - Entradas: filtros temporais, geográficos e por categoria.
    - Saídas: arquivo exportado, dashboard com métricas (visitas ao perfil, buscas mais comuns).
- **Gerenciar permissões / segurança / logs**
    - Descrição: administração de papéis (admin, moderador, usuário), logs de auditoria e políticas de privacidade.
    - Entradas: configurações de usuário e ações administrativas.
    - Saídas: aplicação de permissões, trilhas de auditoria e alertas.




### 6.3 Requisitos Funcionais (RF)

- RF01 — O sistema deve permitir que o usuário (visitante/morador/guia/pesquisador) crie uma conta.

- RF02 — O sistema deve permitir que o usuário autentique-se e recupere senha.

- RF03 — O sistema deve permitir que o usuário edite seu perfil e defina preferências (interesses, acessibilidade, idioma).

- RF04 — O sistema deve permitir que o negócio/proprietário cadastre e atualize a ficha de um ponto turístico (nome, descrição, categoria, endereço, coordenadas, horários, faixa de preço, contatos, acessibilidade, imagens).

- RF05 — O systema deve permitir que o usuário comum sugira edições em pontos turísticos existentes (horários, fotos, avisos).

- RF06 — O sistema deve permitir que o moderador/administrador revise, aprove ou rejeite cadastros e sugestões de edição.

- RF07 — O sistema deve permitir que o proprietário/gestor solicite e receba selo de verificação para seu ponto/negócio mediante comprovação.

- RF08 — O sistema deve permitir busca por pontos turísticos por termo, categoria e filtros (acessibilidade, faixa de preço, horário, distância).

- RF09 — O sistema deve permitir a visualização dos resultados em lista e em mapa com marcadores georreferenciados.

- RF10 — O sistema deve permitir que o usuário visualize a ficha completa do ponto com fotos, avaliações, horários, rota e botão para adicionar ao roteiro ou salvar.

- RF11 — O sistema deve permitir que o usuário crie, edite, salve e exclua roteiros personalizados.

- RF12 — O sistema deve permitir que o motor de recomendação gere roteiros personalizados com base no perfil do usuário, preferências, tempo disponível e horários de funcionamento.

- RF13 — O sistema deve permitir que o usuário aceite, ajuste e compartilhe roteiros (link público/privado).

- RF14 — O sistema deve permitir que os usuários submetam avaliações, comentários e fotos sobre pontos turísticos.

- RF15 — O sistema deve permitir que avaliações e conteúdos submetidos sejam moderados automaticamente e manualmente (filtros de spam/linguagem e revisão humana).

- RF16 — O sistema deve permitir um sistema de reputação/score para usuários colaboradores e negócios.

- RF17 — O sistema deve permitir exportação de dados e geração de relatórios (CSV/GeoJSON/dashboards) para gestores públicos e pesquisadores.

- RF18 — O sistema deve permitir a importação de inventários via arquivos (CSV/GeoJSON) e integração por API para sincronização com bases oficiais.

- RF19 — O sistema deve permitir controle de permissões por papéis (usuário, negócio, moderador, administrador, gestor público) e restrição de acesso a funcionalidades sensíveis.

- RF20 — O sistema deve permitir gravação de logs/auditoria para todas as operações críticas de CRUD (quem, quando, que alteração).

- RF21 — O sistema deve permitir autenticação segura (por exemplo via e-mail/senha com recuperação; possibilidade de autenticação multifator para papéis administrativos).

- RF22 — O sistema deve permitir notificações aos usuários sobre alterações relevantes (aprovação de cadastro, sugestões aceitas, alertas de fechamento/obras).

- RF23 — O sistema deve permitir mecanismos de consenso/crowdsourcing para priorizar alterações (votos ou múltiplas confirmações de usuários verificados).

- RF24 — O sistema deve permitir busca por similaridade (sugerir pontos/roteiros com base em usuários com gostos semelhantes).

- RF25 — O sistema deve permitir configuração e aplicação de regras de negócio para montagem de roteiros (janelas de tempo, tempo estimado de visita, ordem otimizada considerando deslocamento).

- RF26 — O sistema deve permitir o armazenamento de histórico de versões para pontos turísticos e roteiros (possibilitar rollback ou auditoria).

- RF27 — O sistema deve fornecer interfaces administrativas para gestão de conteúdo (moderadores administram relatórios, usuários/problemáticas, duplicidades).

- RF28 — O sistema deve permitir exportação de métricas de uso (número de buscas, roteiros gerados, avaliações, usuários ativos) para suportar análises de adoção e melhoria.

### 6.4 Requisitos Não Funcionais (RNF)

**Desempenho**

- RNF01 — O sistema deve responder a requisições de busca simples em até 300 ms em ambiente de teste (latência média).
- RNF02 — O tempo de geração de um roteiro (recomendações + roteamento) deve ser, em média, inferior a 5 segundos.
- RNF03 — O sistema deve processar uploads de imagens com tempo máximo de 8 segundos por arquivo (tamanho ≤ 5 MB) em conexão padrão.

**Escalabilidade e capacidade**

- RNF04 — A arquitetura deve suportar pelo menos 100 usuários simultâneos sem degradação significativa (latência de busca não mais que 2× do valor médio).
- RNF05 — Deve ser possível escalar horizontalmente os serviços (API, busca, processamento de recomendações) sem alterações significativas no código base.

**Disponibilidade e recuperação**

- RNF06 — O sistema deve garantir disponibilidade mínima de 99% durante o período de avaliação/demonstração.
- RNF07 — Deve existir procedimento de backup diário dos dados principais (banco de dados e arquivos) com retenção mínima de 30 dias.
- RNF08 — O sistema deve recuperar operação crítica (restauração completa) em no máximo 4 horas após falha grave, seguindo procedimentos documentados de disaster recovery.

**Segurança e privacidade**

- RNF09 — Todas as rotas sensíveis devem exigir autenticação; tokens/sessões devem expirar e ser renováveis com segurança.
- RNF10 — Dados sensíveis (credenciais, tokens) devem ser armazenados criptografados em repouso; comunicações entre cliente e servidor devem usar TLS/HTTPS.
- RNF11 — Deve ser implementado controle de acesso baseado em papéis (RBAC) para separar privilégios de usuários, negócios, moderadores e administradores.
- RNF12 — Logs de auditoria devem registrar 100% das operações CRUD críticas com identificador do usuário, timestamp e descrição da ação.

- RNF13 — O sistema deve permitir revogação de consentimento pelo usuário e assegurar conformidade com requisitos básicos de privacidade (ex.: anonimização de dados quando solicitado).

**Usabilidade e acessibilidade**

- RNF14 — Interface principal deve ser responsiva e funcionar em dispositivos móveis e desktop; páginas críticas devem atender boas práticas de performance perceptiva (First Contentful Paint < 2s em conexão padrão).
- RNF15 — A interface deve seguir diretrizes básicas de acessibilidade (WCAG AA como referência mínima) para campos essenciais (navegação, formulários, mapas).
- RNF16 — O fluxo de cadastro de ponto e de criação de roteiro deve permitir conclusão em no máximo 5 passos/etapas claras (meta de tempo: <10 minutos para usuário iniciante).

**Portabilidade e compatibilidade**

- RNF17 — APIs devem expor e consumir formatos padrões (JSON/GeoJSON) para facilitar integração com sistemas externos.
- RNF18 — O front-end deve ser compatível com os navegadores modernos suportados (Chrome, Firefox, Edge e Safari nas versões recentes) e degradar graciosamente em navegadores antigos.

**Dados e governança**

- RNF19 — O sistema deve armazenar metadados de origem para cada registro (origem: usuário, importação, API; timestamp; versão) para permitir rastreabilidade.
- RNF20 — Taxonomia e campos essenciais (categoria, horários, acessibilidade) devem ter validações e dicionário de dados documentado; alterações na estrutura devem ser versionadas.
- RNF21 — Deve existir mecanismo de verificação/qualidade que permita marcar registros com status (p.ex., verificado, pendente, rejeitado) e relatórios periódicos de qualidade de dados.

**Manutenção e operação**

- RNF22 — O código deve ser acompanhado por testes automatizados: cobertura mínima 60% para backend crítico e testes end-to-end cobrindo fluxos principais (cadastro, busca, geração de roteiro).

- RNF23 — Deve existir documentação técnica básica (endpoints API, modelos de dados, procedimentos de deploy e rollback) atualizada e disponível para a equipe.
- RNF24 — Processos de deploy devem suportar rollback automático em caso de falhas críticas detectadas pós-deploy.

**Monitoramento e métricas**

- RNF25 — Deve ser implementado monitoramento de performance e saúde (métricas de latência, erros 5xx, uso de CPU/memória) com alertas configurados para anomalias críticas.
- RNF26 — Métricas de uso (número de buscas, roteiros gerados, cadastros feitos, avaliações) devem ser coletadas e exportáveis para análise (CSV/ dashboards) com agregação mínima diária.

**Teste e validação**

- RNF27 — Antes de qualquer release em ambiente de demonstração, deve ser executado um teste de carga com perfil representativo (pelo menos 100 usuários simultâneos) e relatório documentado.
- RNF28 — Plataforma deve passar por testes de segurança básicos (scans de vulnerabilidade automatizados) e correção de falhas críticas antes da defesa/demonstração.

### 6.5 Regras de Negócio (RB)

RB01 — Acesso restrito por autenticação

- Regra: Apenas usuários autenticados podem realizar ações que alterem dados persistentes (criar/editar/excluir pontos, enviar sugestões, criar roteiros, avaliar).
- Exceção: visualização básica de pontos e buscas pode ser permitida a usuários anônimos.

RB02 — Diferenciação por papéis (RBAC)

- Regra: Operações sensíveis só podem ser executadas por papéis autorizados (ex.: aprovadores/moderadores para publicar pontos verificados; administradores para configurações do sistema).
- Ação: O sistema deve verificar papel/permisões antes de aceitar a ação.

RB03 — Verificação para selo “verificado”

- Regra: Apenas proprietários/gestores comprovados ou moderadores podem atribuir o selo “verificado” a um ponto/negócio.
- Condição: solicitação deve incluir prova documental ou validação por API oficial; poderá passar por revisão manual.
- Ação: Aplicar selo visível na interface e registrar evidência no histórico.

RB04 — Sugestões crowdsourced com níveis de confiança

- Regra: Sugestões de edição submetidas por usuários não verificados entram em fila de revisão; edições submetidas por proprietários verificados ou múltiplos usuários confiáveis podem ser aplicadas automaticamente.
- Ação: Sistema aplica regra de confiança (p.ex., thresholds de número de confirmações ou reputação do usuário) para decidir publicação automática ou revisão manual.

RB05 — Prevenção de duplicidade geoespacial

- Regra: Ao cadastrar um novo ponto, o sistema deve checar duplicidade por nome e proximidade geográfica; se encontrar potencial duplicado, apresentar aviso e pedir confirmação antes de criar novo registro.
- Ação: Permitir vincular registros como “mesmo local” ou criar registro separado com indicação de possível duplicidade.

RB06 — Campos obrigatórios mínimos para publicação

- Regra: Um ponto só pode ser publicado como “ativo” se atender campos obrigatórios (nome, coordenadas, categoria e horário de funcionamento ou indicação de abertura contínua).
- Ação: Rejeitar publicação e notificar cadastrado com lista de campos faltantes.

RB07 — Validação de horários e janelas de funcionamento

- Regra: Roteiros gerados não devem incluir visitas a pontos fora de suas janelas de funcionamento; o motor de recomendação deve respeitar horários e restrições temporais.
- Ação: Excluir candidatos do roteiro fora do horário, ou sugerir alternativa/alerta com justificativa.

RB08 — Política de edição concorrente

- Regra: Quando múltiplos usuários editam a mesma ficha simultaneamente, o sistema deve bloquear a edição (lock pessimista) ou alertar sobre conflito e criar versão diferenciada (merge/rollback).
- Ação: Registrar conflito no histórico e pedir resolução manual se necessário.

RB09 — Retenção e auditoria de alterações

- Regra: Todas as alterações que afetam dados publicados devem ser auditadas (quem, quando, antes/depois) e mantidas por no mínimo 30 dias para fins de compliance e rollback.
- Ação: Expor histórico de versões e permitir restauração por moderadores.

RB10 — Limites de taxa para operações sensíveis

- Regra: Aplicar limites (rate limits) a operações que podem ser abusadas (p.ex., submissão de sugestões, criação de contas em massa, uploads de imagens) para evitar spam/abuso.
- Ação: Bloquear temporariamente IP/usuário em caso de exceder limite e notificar administrador se padrão suspeito persistir.

### 6.6 Fora do Escopo

1. Venda direta de ingressos e processamento de pagamentos

- O sistema não atuará como plataforma de pagamento ou gateway para venda direta de ingressos; integrações com serviços de reserva/pagamento poderão ser previstas, mas pagamentos em si serão redirecionados para provedores externos.

1. Gestão operacional de reservas (pós-venda) e CRM completo

- Não será responsabilidade do sistema gerenciar check-ins, cancelamentos detalhados, faturamento ou processos completos de CRM empresarial; funcionalidades de contato/reserva serão limitadas a encaminhamento/links para sistemas do negócio.

1. Monitoramento em tempo real de condições físicas (sensores IoT, câmeras)

- O projeto não incluirá integração com sensores IoT, câmeras de vigilância ou captura em tempo real de condições ambientais (por

exemplo, lotação ao vivo, câmeras de tráfego). Alertas e fechamentos poderão ser atualizados via contribuições de usuários e órgãos oficiais.

1. Funcionalidades avançadas de e-commerce ou marketplace

- Não será desenvolvido um marketplace completo (carrinho, checkout, gestão de pedidos, logística). Promoções e destaques podem ser exibidos no perfil dos negócios, mas transações não serão processadas pela plataforma.

1. Tradução automática completa e localização multilíngue avançada

- O escopo não inclui tradução automática em tempo real e suporte completo a múltiplos idiomas em todas as interfaces; haverá suporte a campos de conteúdo em diferentes idiomas e interface em um idioma principal (definido no projeto), com possibilidade de expansão futura.

1. Serviços de atendimento ao cliente humano 24/7

- Não haverá central de atendimento humano integrada ao produto operando em tempo integral; notificações e mensagens automáticas serão suportadas, e contatos serão encaminhados aos responsáveis locais conforme configuração.

1. Garantia de veracidade absoluta para todas as informações

- Apesar de mecanismos de verificação e moderação, o sistema não garante que todas as informações estejam sempre 100% corretas ou atualizadas; discrepâncias entre fontes poderão ocorrer e serão sinalizadas para revisão humana.

1. Suporte para integração com todos os sistemas de terceiros possíveis

- Integrações serão propostas via API e importação CSV/GeoJSON, mas não haverá suporte out-of-the-box para todos os sistemas do mercado; conexões específicas poderão requerer desenvolvimento adicional ou acordos com provedores.

1. Inteligência artificial de alta complexidade fora do escopo

- O projeto não incluirá modelos de IA de grande escala ou treinamento massivo customizado (por exemplo, modelos de linguagem treinados do zero); o motor de recomendação será uma solução híbrida simples (regras + algoritmos leves) adequada ao escopo acadêmico.

1. Funcionalidades legais e administrativas complexas

- O sistema não assumirá responsabilidades legais ou administrativas dos negócios cadastrados (ex.: certificações, licenças, fiscalização); ele

apenas exibirá informações declaradas e/ou verificadas conforme regras internas.

# 7 Fluxos e Comportamento do Sistema

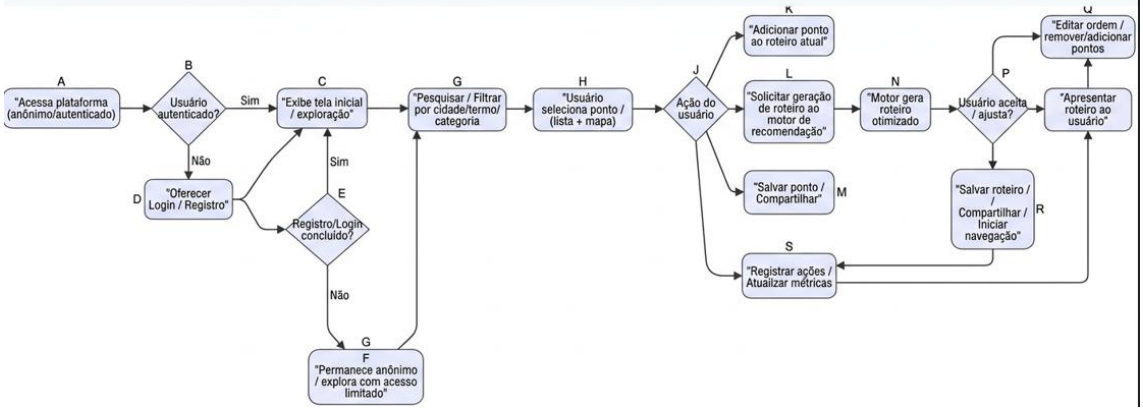
## 7.1 Fluxo Principal do Usuário

### - fluxogramas

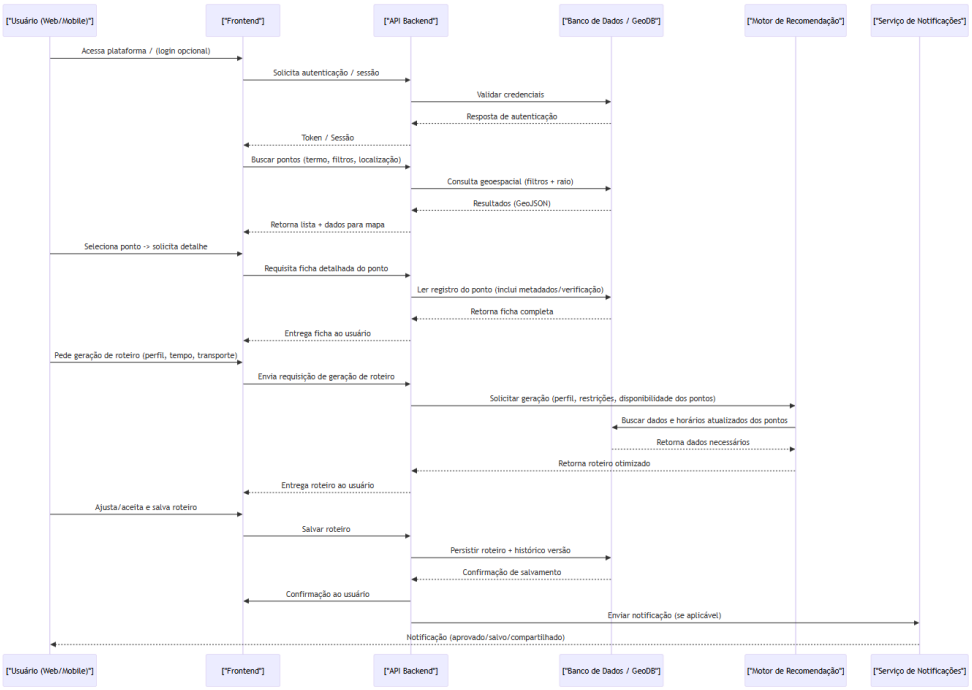


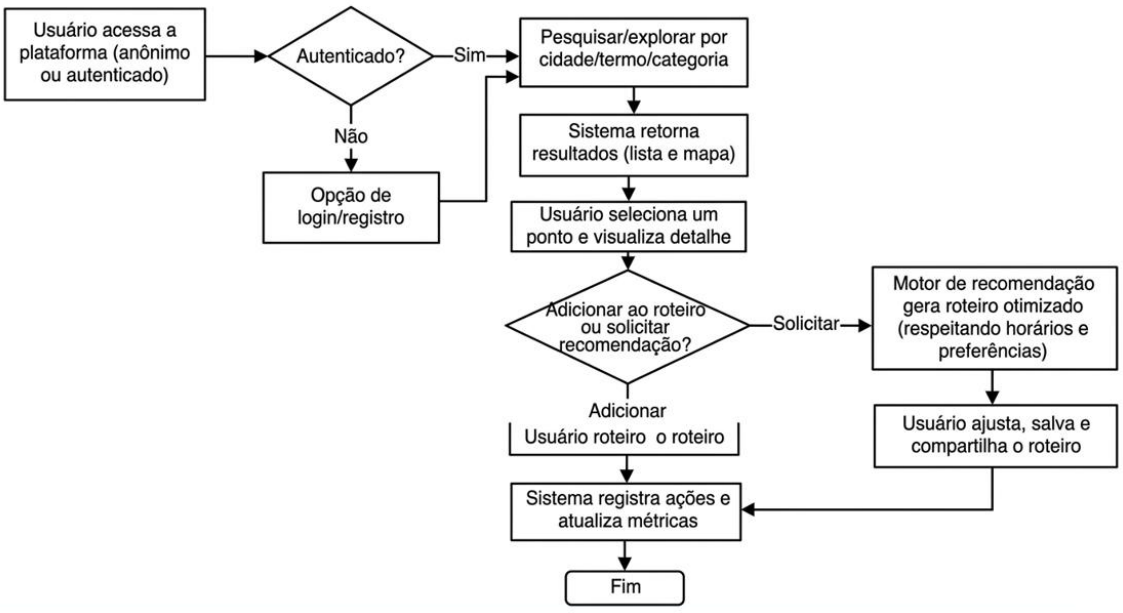
### - diagramas de atividades

```
flowchart LR
    A. "Acessa plataforma (anônimo/autenticado)"
    B. "Usuário autenticado?"
    C. "Exibe tela inicial / exploração"
    D. "Oferecer Login / Registro"
    K. "Adicionar ponto ao roteiro a/ lileteira ao motor de recomendação"
    L. "Solicitar geração de roteiro ao motor de renonmendação"
    N. "Motor gera ncsra roteiro otimizado"
    O. "Apresentar roteiro ao usuário"
    P. "Salvar roteiros / / Atualizar métricas"
```




### - diagramas de sequência




---

## 7.2 Fluxos Alternativos

**- erros**

**- cancelamentos**

**- exceções**

AF01 — Falha de autenticação (credenciais inválidas)

- Condição: usuário tenta logar com e-mail/senha incorretos ou conta inexistente.
- Fluxo:

    i. Frontend envia credenciais à API.

    ii. API valida no banco e retorna falha de autenticação.

    iii. Frontend exibe mensagem clara: "E-mail ou senha inválidos. Verifique e tente novamente."

    iv. Após 3 tentativas falhas consecutivas, sistema aplica bloqueio temporário (ex.: 10–30 minutos) e exibe: "Muitas tentativas. Tente novamente em X minutos ou recupere a senha."

- Ações do sistema: registrar tentativas no log de segurança (usuário/IP/timestamp), disparar alerta se padrão suspeito (múltiplos bloqueios) e oferecer link de recuperação de senha.
- Recuperação: usuário usa fluxo de recuperação por e-mail para redefinir senha.

AF02 — Timeout / Latência alta na busca

- Condição: a consulta de busca excede o tempo limite (por carga ou problema de infra).
- Fluxo:

    i. Frontend envia solicitação de busca; API encaminha para serviço de busca/DB.

    ii. Se não houver resposta dentro do timeout configurado (ex.: 5 s), API retorna erro de timeout.

    iii. Frontend exibe mensagem: "Não foi possível completar a busca no momento. Tente novamente." e mostra opção de recarregar ou reduzir filtros.

- Ações do sistema: registrar evento (erro, latência), incrementar métrica de SLI/SLO, tentar fallback (ex.: buscas em cache com dados mais antigos) se disponível.
- Recuperação: usuário pode reexecutar a busca; equipe técnica analisa alertas e logs.

AF03 — Erro ao carregar imagens (upload falha)

- Condição: falha de upload por conexão interrompida, arquivo excede limite ou formato inválido.
- Fluxo:

    i. Usuário faz upload de imagem na ficha do ponto ou avaliação.

    ii. Serviço de upload responde com erro (413 — payload too large; 415 — unsupported media type; ou 5xx).

    iii. Frontend exibe mensagem específica:

    - Arquivo muito grande: "A imagem excede 5 MB. Reduza o tamanho e tente novamente."
    - Formato inválido: "Formato não suportado. Use JPG/PNG."
    - Erro temporário: "Falha no upload. Tente novamente."
- Ações do sistema: não persiste arquivo parcial; registra tentativa e motivo; aplica retry automático em falhas temporárias (até N vezes).
- Recuperação: permitir novo envio, com instruções e validação no cliente para evitar repetição do erro.

AF04 — Conflito de edição concorrente (duas pessoas editam a mesma ficha)

- Condição: usuário A abre edição da ficha; usuário B também abre e salva alterações quase simultâneas.
- Fluxo:

- Ao iniciar edição, sistema pode aplicar lock otimista (versionamento) ou pessimista (bloqueio).
- Se for lock otimista: quando o usuário B submete, backend detecta que a versão atual mudou desde que o usuário A começou a editar.
- API retorna conflito (HTTP 409) e frontend mostra: "O conteúdo foi alterado por outro usuário. Deseja ver as alterações e mesclar?"
- Ações do sistema: armazenar ambas as versões no histórico, oferecer comparação (diff) e opções: aceitar versão mais recente, mesclar manualmente ou manter ambas (criar rascunho). Registrar evento de conflito.
- Recuperação: usuário resolve o conflito via UI; moderador pode intervir se necessário.

AF05 — Sugestão de edição marcada como potencialmente conflitante / spam

- Condição: submissão de edição com conteúdo suspeito (links maliciosos, linguagem imprópria) ou muitas sugestões repetitivas por mesma conta/IP.
- Fluxo:

- Sistema aplica filtros automáticos (detecção de spam/ofensa).
- Se detectado, sugestão vai para fila de moderação (estado “pendente/flagged”) e autor recebe aviso: "Sua sugestão será analisada."
- Moderador revê e aprova/rejeita/solicita mais informações.
- Ações do sistema: bloquear ações repetidas conforme regras de rate limit, registrar logs e notificar moderadores.
- Recuperação: após revisão, alteração publicada ou revertida; usuário recebe feedback.

AF06 — Falha ao gerar roteiro (erro no motor de recomendação)

- Condição: o motor de recomendação falha por erro interno, falta de dados, ou timeout.
- Fluxo:

- Frontend solicita geração; API encaminha ao motor.
- Se o motor responde com erro ou timeout, API retorna mensagem de falha.
- Frontend exibe: "Não foi possível gerar o roteiro. Tente novamente ou crie manualmente."
- Ações do sistema: registrar stacktrace/erro, oferecer fallback: gerar roteiro simples por heurística (ex.: top N por proximidade) ou usar último roteiro em cache, e notificar equipe/monitoramento.
- Recuperação: usuário pode tentar novamente; fallback garante alguma resposta útil enquanto a falha é resolvida.

AF07 — Ponto não elegível para publicação (campos obrigatórios ausentes)

- Condição: usuário tenta publicar ponto sem preencher campos mínimos.
- Fluxo:

- Ao submeter, backend valida campos obrigatórios.
- Se faltar algum campo, API retorna erro de validação com lista de campos faltantes.
- Frontend destaca os campos faltantes e mostra mensagem: "Para publicar, preencha: nome, coordenadas, categoria e horário."
- Ações do sistema: prevenir publicação; salvar rascunho opcional; registrar tentativa de publicação incompleta.
- Recuperação: usuário preenche campos e reenvia.

AF08 — Cancelamento durante criação de roteiro

- Condição: usuário inicia criação/edição de roteiro e cancela (fecha a janela ou “voltar”).
- Fluxo:

    i. Frontend detecta intenção de sair sem salvar e exibe diálogo: "Deseja salvar rascunho antes de sair?" com opções: Salvar rascunho / Descarta / Continuar editando.

    ii. Se escolher salvar, sistema persiste rascunho; se descartar, alterações locais são perdidas.
- Ações do sistema: salvar rascunho com metadados (quem, quando) quando solicitado; registrar evento de cancelamento.
- Recuperação: usuário pode retomar rascunho a qualquer momento.

AF09 — Importação de inventário com formato inválido / erro na integração

- Condição: importação CSV/GeoJSON mal formatada ou falha de autenticação em integração externa.
- Fluxo:

    i. Upload/integração inicia; parser detecta erro de formato ou campo obrigatório ausente.

    ii. Sistema interrompe processamento e gera relatório de importação com erros por linha/registo.

iii. Frontend/console exibe resumo: número de registros processados, sucessos, falhas e motivos.

- Ações do sistema: não importar registros inválidos; permitir download do relatório de erro; oferecer template de importação e instruções de correção.
- Recuperação: operador corrige arquivo e reimporta; integração externa requer reautenticação se falha por credenciais.

AF10 — Remoção/acesso de dados sensíveis solicitada (GDPR/privacidade)

- Condição: usuário solicita exclusão/anônimização de dados pessoais.
- Fluxo:

i. Sistema exibe formulário de solicitação; API autentica e valida a identidade do solicitante.

ii. Após validação, sistema remove/anonimiza dados pessoais conforme regra e registra ação no log de auditoria.

iii. Confirmação enviada ao usuário: "Seus dados foram removidos/anonimizados."

- Ações do sistema: manter registro não pessoal para métricas agregadas; enviar notificação ao responsável; atualizar índice de busca.
- Recuperação: ações em geral são irreversíveis; políticas de retenção e exceções (por exigência legal) devem ser documentadas.

# 8 Mockups e Experiência do Usuário (UX)

As imagens desta seção são **capturas do sistema implementado**, não desenhos feitos antes do código: as telas foram prototipadas diretamente em React, sobre um conjunto de componentes e uma paleta próprios (verde-petróleo para ação principal, coral para destaque e confirmação). A interface é responsiva e funciona no computador e no celular. As telas de busca e de detalhe mostram **pontos reais**, importados do OpenStreetMap (seção 9.5); os roteiros e os nomes de usuários das demais telas são de demonstração, criados pelos testes de ponta a ponta.

Princípios de experiência que orientaram o desenho das telas:

- **Descoberta sem cadastro:** busca, mapa e detalhes de pontos são públicos; o login só é exigido para salvar, avaliar ou montar roteiros.
- **Feedback imediato:** mensagens de sucesso e de erro em português, estados de carregamento e confirmação antes de ações destrutivas.
- **Um caminho principal curto:** da busca ao roteiro organizado em quatro passos (seção 8.3).
- **Funcionalidades por papel:** cada perfil (visitante, morador, negócio, gestor, moderador, administrador) só vê os menus que pode usar.
- **Acessibilidade e privacidade como dado de primeira classe:** o perfil guarda necessidades de acessibilidade, e a LGPD é tratada no próprio Perfil.

### 8.1 Fluxo de Navegação

O diagrama abaixo mostra como a pessoa navega entre as telas. Ele está organizado em três faixas: o que qualquer visitante acessa sem login; o que fica disponível após o login; e as áreas reservadas a negócios, gestores, moderadores e administradores. As setas tracejadas indicam o compartilhamento de um roteiro por link público.

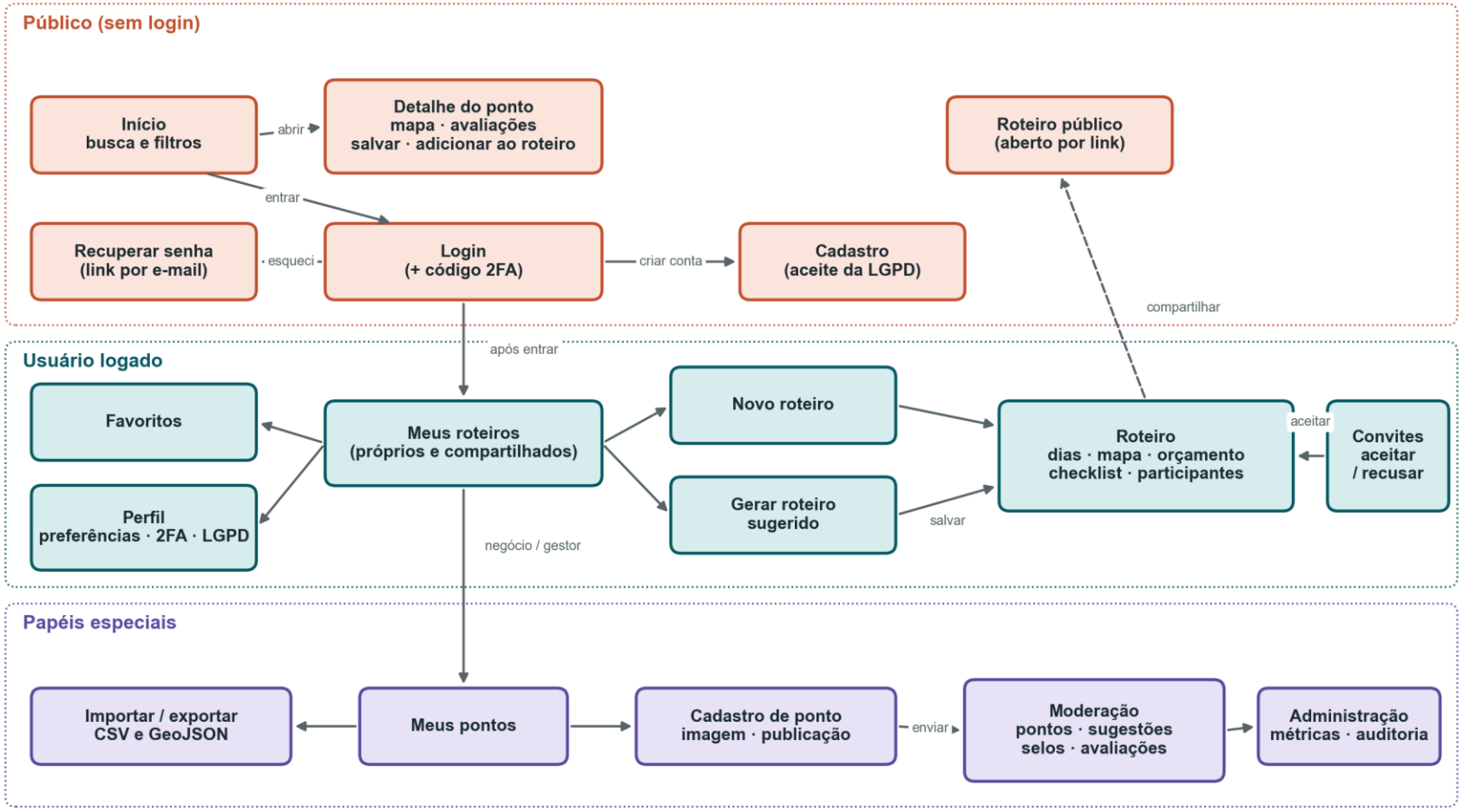



*Figura 1 — Fluxo de navegação do NomadPlan, por tipo de acesso.*

### 8.2 Wireframes ou Mockups das Telas

Para cada tela estão a imagem, a funcionalidade e as principais ações do usuário. A numeração entre parênteses indica os requisitos funcionais da seção 6.3 atendidos.

**Tela inicial e busca (RF08, RF09)**

Página de entrada do sistema. Reúne a busca por cidade e termo, os atalhos por categoria, a opção de filtros adicionais e a alternância entre lista e mapa. Para quem está logado, exibe recomendações calculadas a partir dos interesses declarados e do que usuários de gosto parecido salvaram.

- Buscar por cidade, nome ou descrição e filtrar por categoria, faixa de preço, acessibilidade, nota, horário (“aberto agora”) e distância.
- Alternar entre **Lista** e **Mapa** e salvar um ponto nos favoritos pelo coração do cartão.
- Abrir o detalhe de um ponto.




*Figura 2 — Tela inicial com busca, categorias, recomendações e resultados.*

**Tela inicial no celular (RNF, uso em dispositivos móveis)**

A mesma página em tela de 390 px de largura: os campos empilham e o menu colapsa, mantendo todas as funções.




*Figura 3 — Versão para celular da tela inicial.*

**Detalhe do ponto turístico (RF10, RF05)**

Ficha completa do ponto: imagem, categoria, selo de verificação (quando concedido), endereço, horário, faixa de preço, acessibilidade, mapa com o local e avaliações de outros usuários. Nos pontos importados, a ficha mostra o crédito da foto (autor e licença) e a fonte dos dados, e convida quem está logado a informar o que falta, como o horário.

- Ver a localização no mapa e abrir a rota (“Como chegar”).
- Quando logado: salvar nos favoritos, adicionar a um roteiro, avaliar com nota e comentário e **sugerir edição** de dados desatualizados.

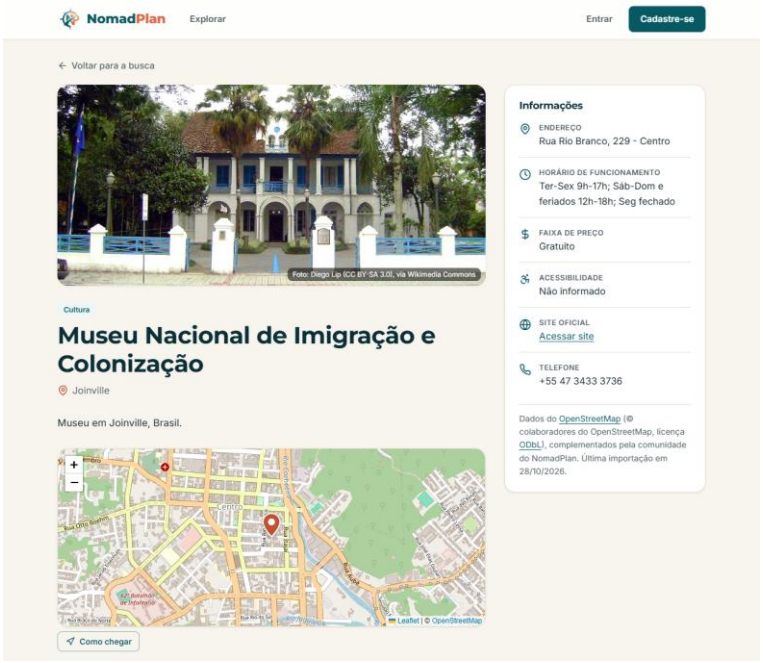



*Figura 4 — Detalhe de um ponto turístico importado do OpenStreetMap, com foto do Wikimedia Commons e mapa.*

### Entrar (RF02)

Formulário de e-mail e senha, com link para recuperação de senha (um e-mail com link de uso único e 15 minutos de validade) e para o cadastro. Se a conta tem verificação em duas etapas, o sistema pede em seguida o código de 6 dígitos.

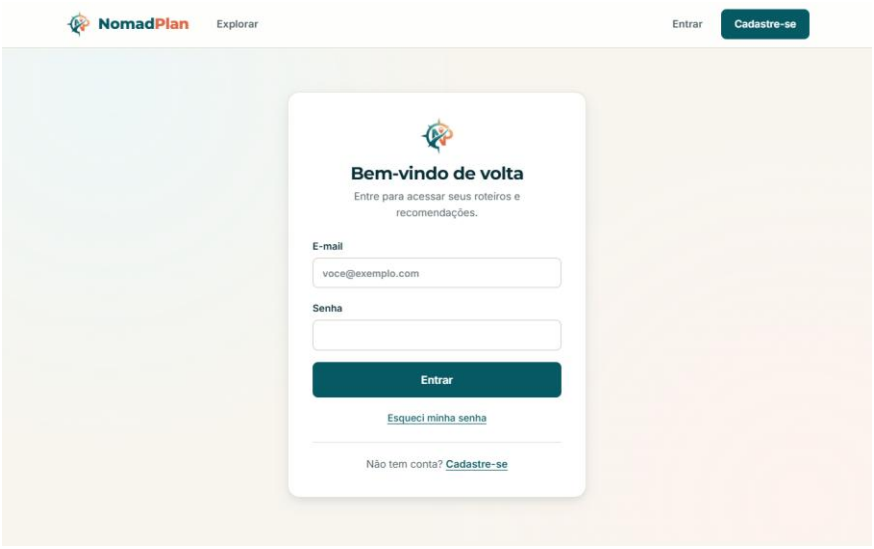



*Figura 5 — Tela de login.*

### Criar conta (RF01)

Cadastro com nome, e-mail, senha (mínimo de 8 caracteres), cidade base opcional e o tipo de uso — visitante/turista, morador, negócio turístico ou

gestor/pesquisador. O cadastro só é concluído com o **aceite da Política de Privacidade** (LGPD).

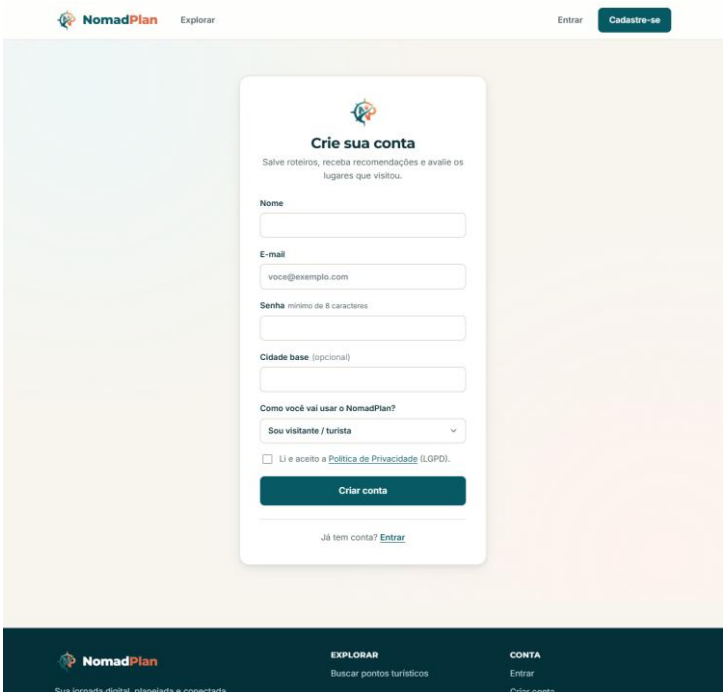



*Figura 6 — Tela de cadastro.*

**Roteiro por dia (RF11, RF13)**

Tela central do planejamento. Mostra o roteiro dividido por dia, com o mapa de cada dia, o tempo e a distância estimados entre as paradas e o custo previsto. As paradas podem ser reordenadas, movidas entre dias, removidas, ou reorganizadas automaticamente pelo botão **Otimizar ordem**, que calcula um trajeto curto entre os pontos do dia.

- Alternar o mapa entre todos os dias ou um dia específico.
- Adicionar pontos, escolher o dia e anotar custo e observação por parada.
- Compartilhar o roteiro por link público (somente leitura) ou revisar o histórico de versões.

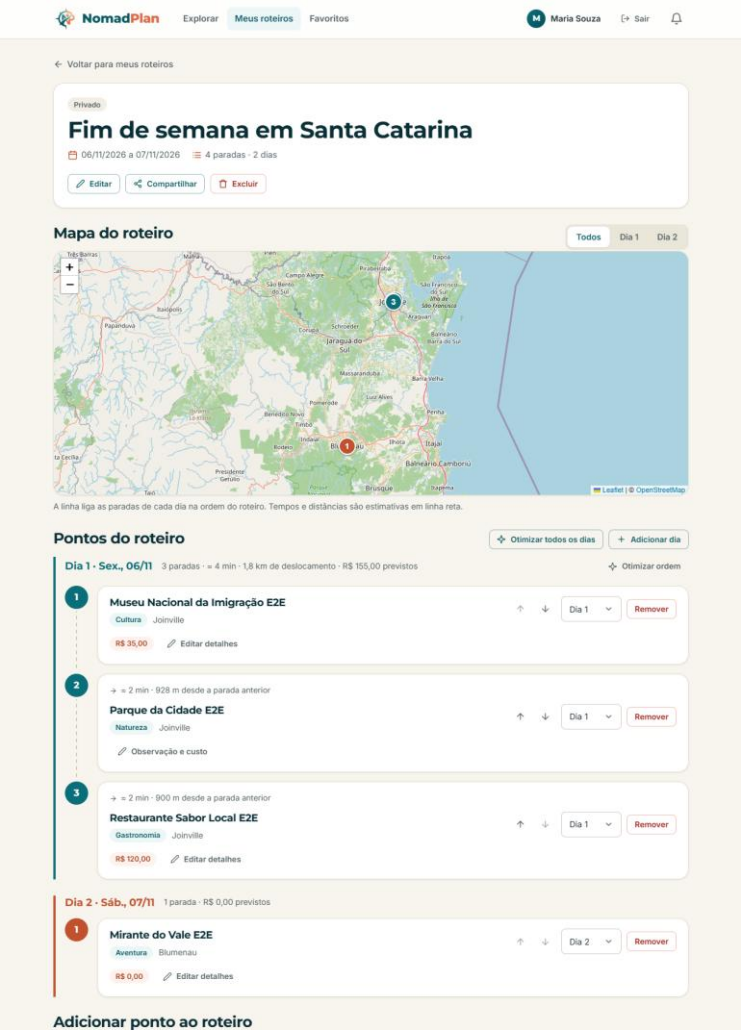



*Figura 7 — Roteiro de dois dias com mapa, paradas e tempos de deslocamento.*

## Orçamento, checklist e notas (RF11)

Abaixo do roteiro, três cartões ajudam a preparar a viagem: o **orçamento** soma os custos informados por dia e compara com um teto opcional; o **checklist** guarda tarefas como “reservar hotel”; as **notas** ficam visíveis apenas para quem participa do roteiro (não aparecem no link público).

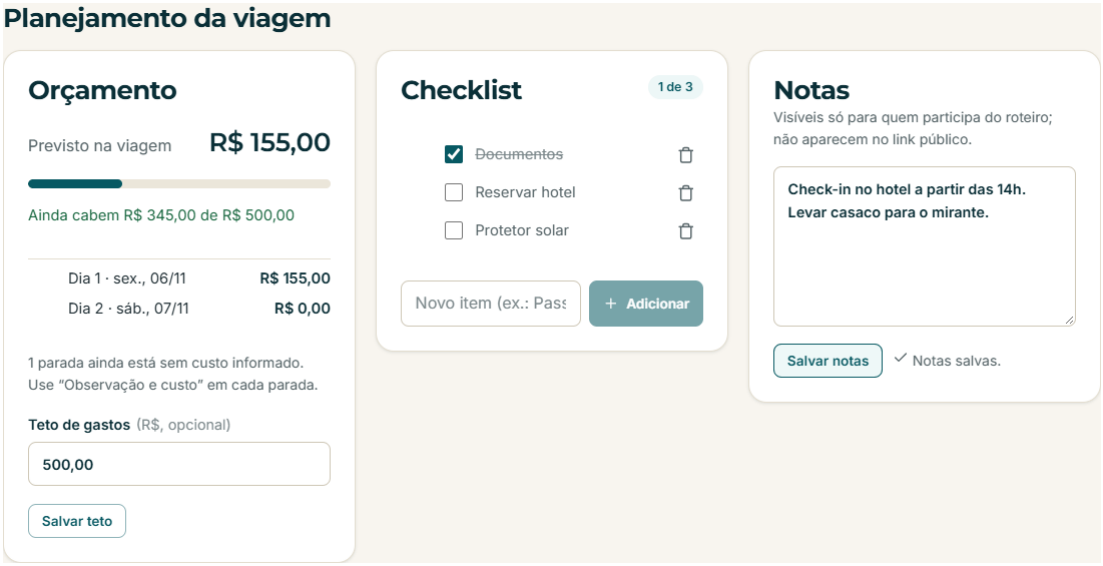



*Figura 8 — Orçamento, checklist e notas da viagem.*

## Roteiro colaborativo e convites (RF13)

O dono convida pessoas por e-mail, escolhendo o papel: **editor** (altera paradas, dias e orçamento) ou **leitor** (apenas acompanha). Enquanto o convite não é aceito, ele aparece como pendente e pode ser cancelado. Quem recebe o convite o encontra na tela *Convites*, onde aceita ou recusa.

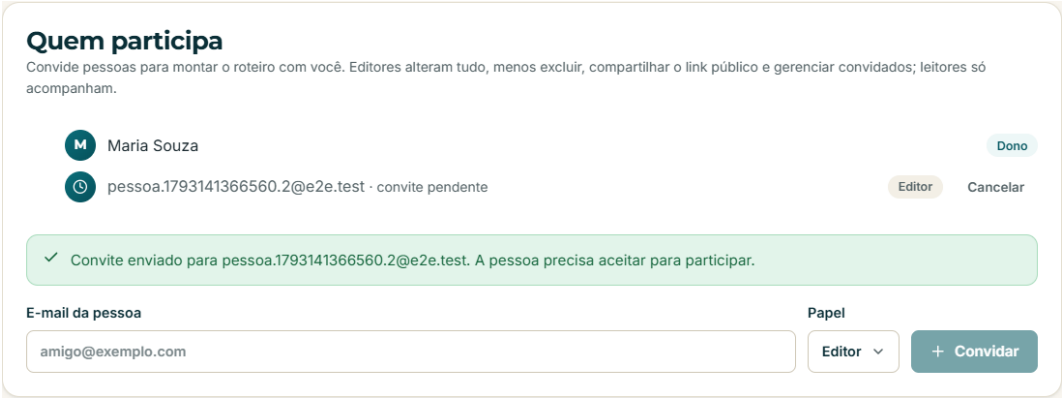



*Figura 9 — Seção "Quem participa" de um roteiro, com convite pendente.*

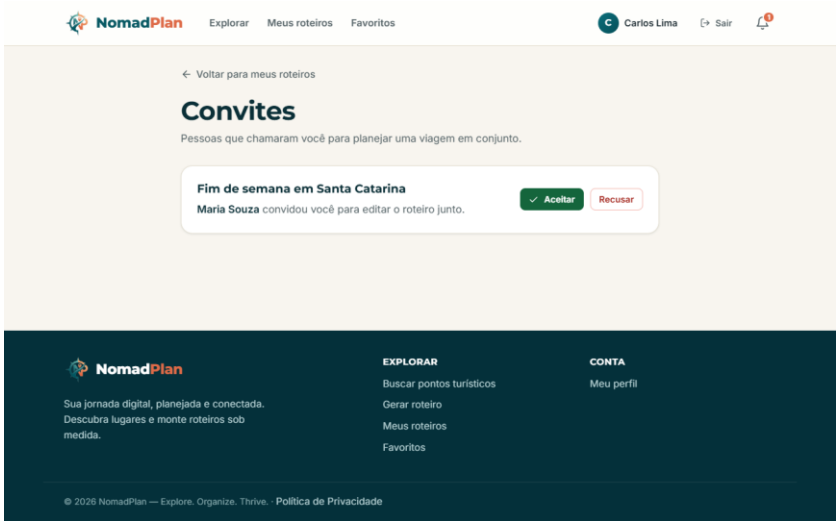



*Figura 10 — Convites recebidos, com as ações Aceitar e Recusar.*

**Gerar roteiro sugerido (RF12)**

O usuário informa a cidade, o tempo disponível, o horário de início, um ponto de partida (opcionalmente a localização atual) e seus interesses. O motor de recomendação monta uma sequência de visitas que respeita o horário de funcionamento de cada local e a distância entre eles, e o resultado pode ser salvo como roteiro.

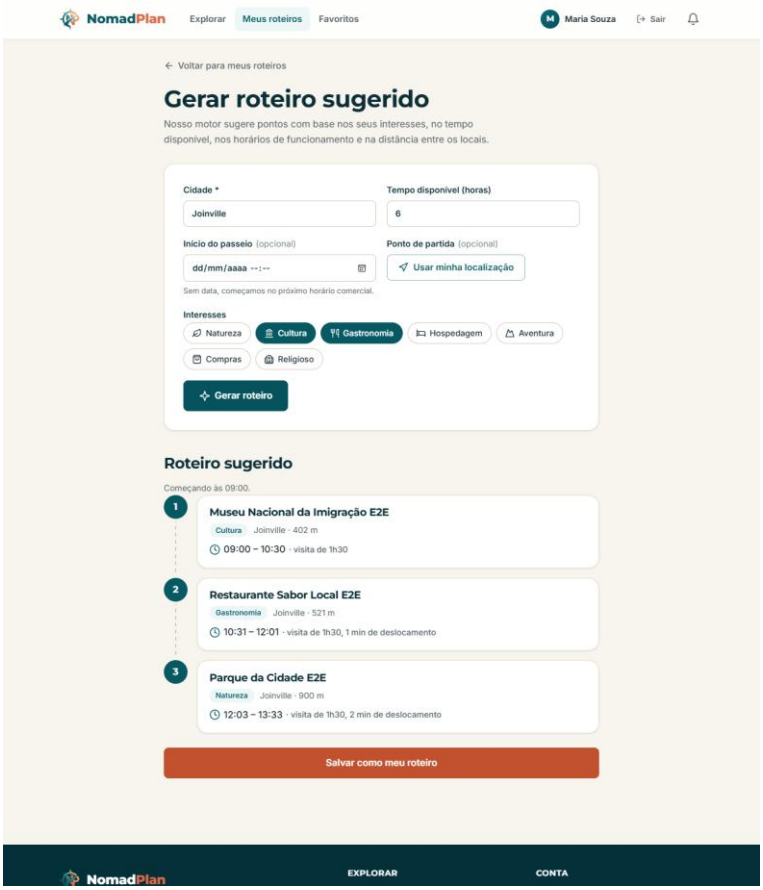



*Figura 11 — Geração de um roteiro sugerido, com horários de cada visita.*

### Cadastro de ponto turístico (RF04)

Formulário usado por negócios, gestores e moderadores para cadastrar e editar pontos: informações básicas, localização por latitude e longitude, horário de funcionamento, faixa de preço, acessibilidade, contatos e imagem. O ponto é salvo como rascunho e **enviado para publicação**, passando pela moderação.

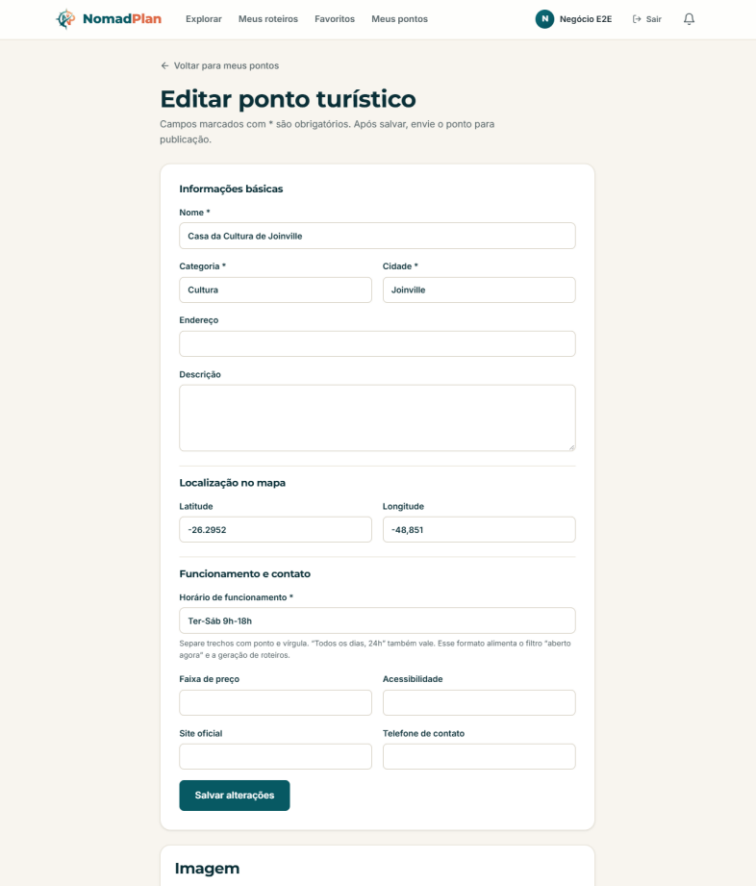



*Figura 12 — Formulário de cadastro e edição de ponto.*

### Ferramentas de inventário (RF04)

Exportação do inventário em CSV ou GeoJSON e importação em lote (até 2 MB) a partir de modelos oferecidos pelo próprio sistema. Os pontos importados entram como rascunho e seguem o fluxo normal de publicação; há uma opção para ignorar a detecção de duplicados.




*Figura 13 — Exportação e importação de pontos em CSV e GeoJSON.*

**Moderação (RF06, RF07)**

Painel do moderador, com a fila de pontos aguardando publicação, sugestões de edição (mostrando o campo alterado), solicitações de selo verificado e avaliações pendentes.

- Aprovar ou rejeitar cada item; ao rejeitar, o motivo é registrado e o autor é notificado.

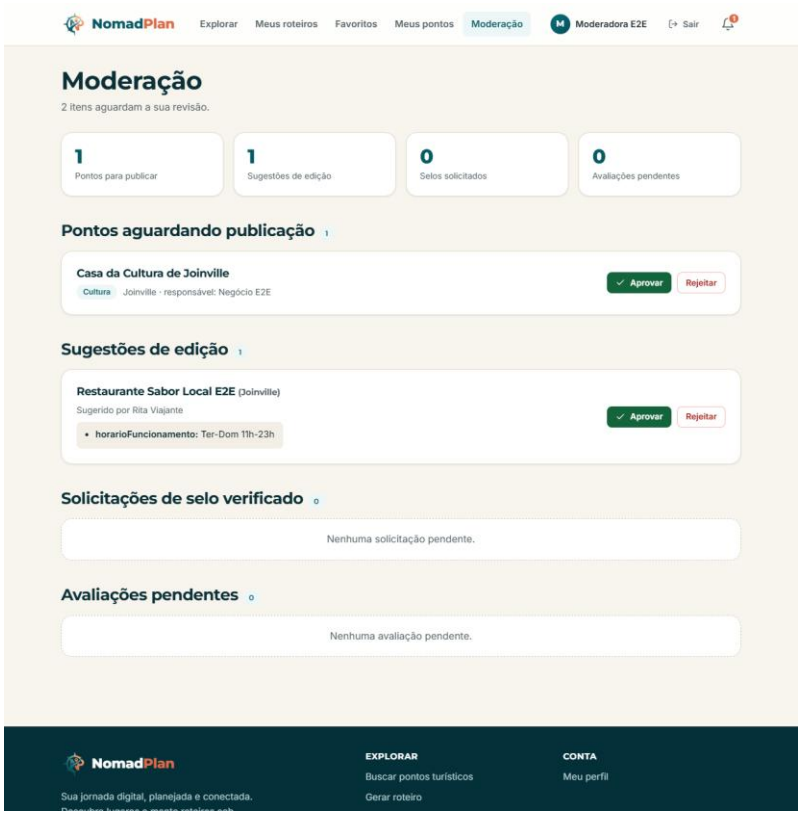



*Figura 14 — Painel de moderação.*

**Administração (RF20)**

Métricas de uso (usuários, pontos, roteiros, avaliações), gráfico de uso diário com exportação em CSV, gestão de usuários e papéis (alterar papel, desativar conta) e consulta ao log de auditoria.

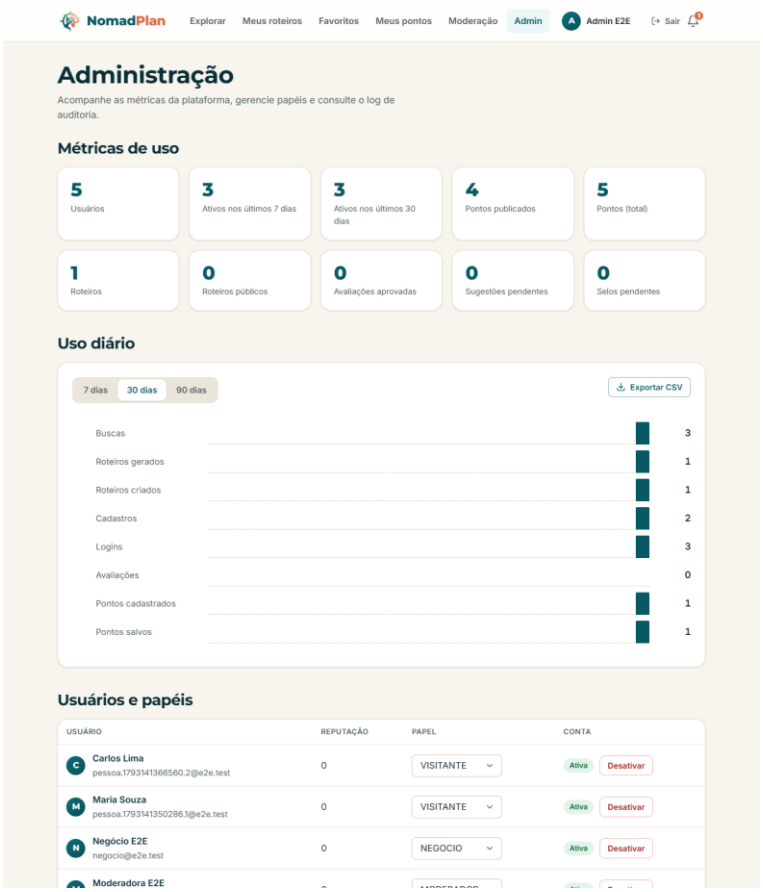



*Figura 15 — Painel administrativo com métricas e gestão de usuários.*

### Perfil, preferências e privacidade (RF03)

O usuário edita nome, cidade, idioma, interesses e necessidades de acessibilidade, ativa a **verificação em duas etapas** lendo um QR code no aplicativo autenticador e, na parte inferior, baixa seus dados em JSON ou anonimiza a própria conta (seção 10.1).

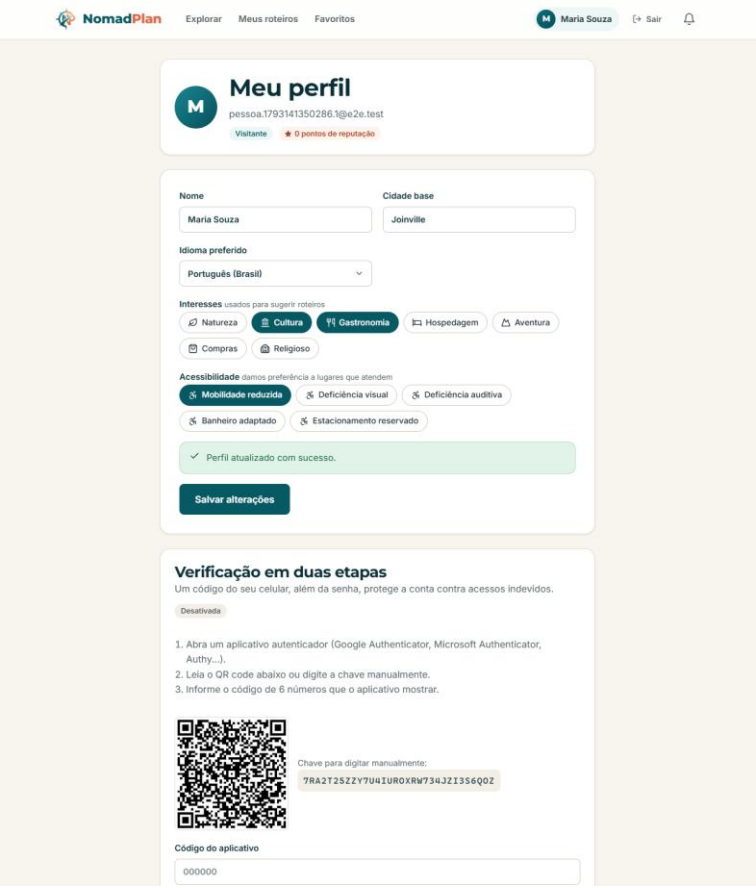



*Figura 16 — Perfil, preferências e verificação em duas etapas.*

### 8.3 Fluxo de Interação do Usuário

O fluxo mais importante do sistema leva uma pessoa de turismo da entrada no sistema a um roteiro organizado por dia. A sequência abaixo resume as telas percorridas:

- **Entrar** — a pessoa cria uma conta ou entra com e-mail e senha (e código de 2 etapas, se ativado).
- **Buscar e filtrar** — escolhe a cidade, aplica categorias e filtros e alterna entre lista e mapa.
- **Ver o ponto e adicionar ao roteiro** — abre a ficha do local, confere horário, mapa e avaliações e adiciona o ponto a um roteiro novo ou existente.
- **Organizar o roteiro** — distribui as paradas pelos dias, ajusta a ordem (manualmente ou pela otimização), confere tempos e custos e, se quiser, compartilha por link ou convida colaboradores.

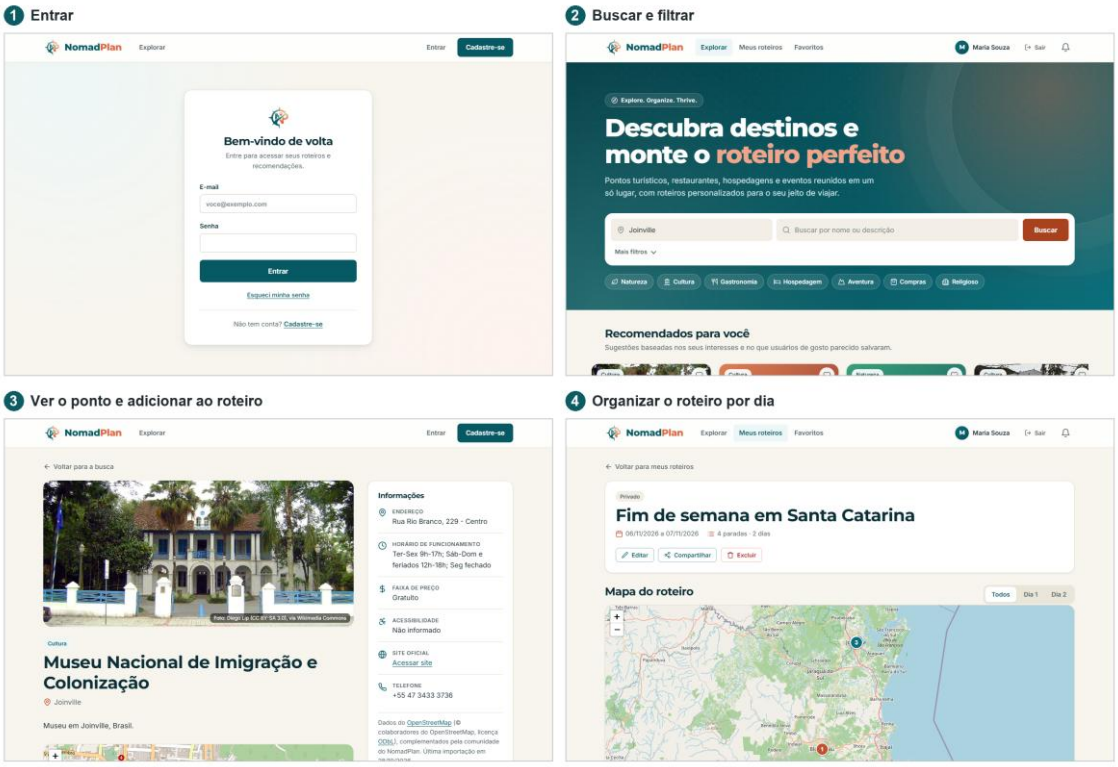



*Figura 17 — Sequência de telas do fluxo principal: entrar, buscar, ver o ponto e organizar o roteiro.*

Alternativa de atalho: na tela **Gerar roteiro**, o usuário informa cidade, tempo e interesses e recebe um roteiro pronto, que pode ser salvo e depois ajustado na mesma tela de organização.

### 8.4 Feedback Inicial de Usuários (Opcional)

Até a entrega do RFC não houve pesquisa formal de usabilidade com usuários externos; por isso esta subseção não apresenta depoimentos. A validação das telas foi feita de duas formas: (i) **74 testes de ponta a ponta** em navegador real, que percorrem os fluxos descritos nesta seção (cadastro, busca, moderação, roteiros por dia, planejamento, colaboração, dados importados, perfil e administração) e conferem as mensagens exibidas; e (ii) revisões internas da equipe a cada módulo entregue. A apresentação final será usada para coletar comentários da banca e de convidados, que alimentarão a próxima iteração.

# 9 Arquitetura do Sistema

Esta seção descreve como o sistema foi construído: contexto, containers e componentes (modelo C4), modelo de dados, módulos principais e as tecnologias escolhidas.

## 9.1 Diagrama C4

### Nível 1 — Diagrama de Contexto

O NomadPlan é uma plataforma web única, usada por seis perfis de pessoas: visitante/turista, morador, negócio/gestor de pontos, gestor de turismo, moderador e administrador. Ele depende de quatro sistemas externos: um **servidor SMTP** (e-mails de recuperação de senha e de convites); o **OpenStreetMap**, que fornece as imagens do mapa, carregadas direto pelo navegador, e a base aberta de onde vêm os pontos turísticos, baixada pela Overpass API; o **Wikimedia Commons**, de onde vêm as fotos dos pontos importados; e um **aplicativo autenticador** do próprio usuário, que gera os códigos TOTP da verificação em duas etapas.

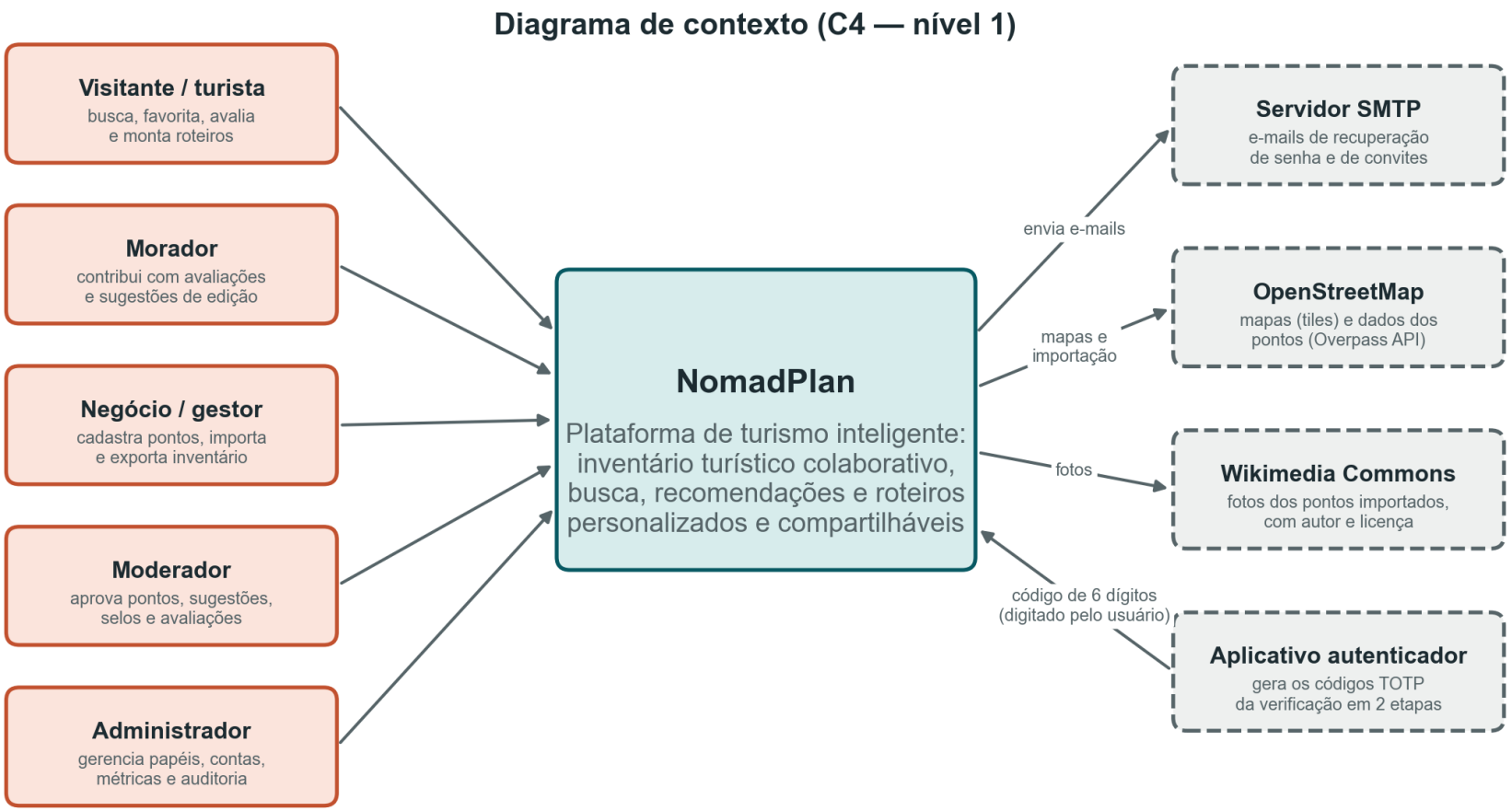



*Figura 18 — Diagrama de contexto (C4, nível 1).*

**Nível 2 — Diagrama de Containers**

Dentro do sistema há quatro containers: a **aplicação web** (SPA em React, entregue como arquivos estáticos), a **API REST** (Node.js/Express), o **banco de dados** (PostgreSQL com a extensão PostGIS) e a pasta de **arquivos enviados** (imagens de pontos e fotos de avaliações), servida pela própria API em modo somente leitura. O navegador fala com a API em JSON sobre HTTPS, com o token de sessão (JWT) no cabeçalho; a API acessa o banco por SQL via Prisma.

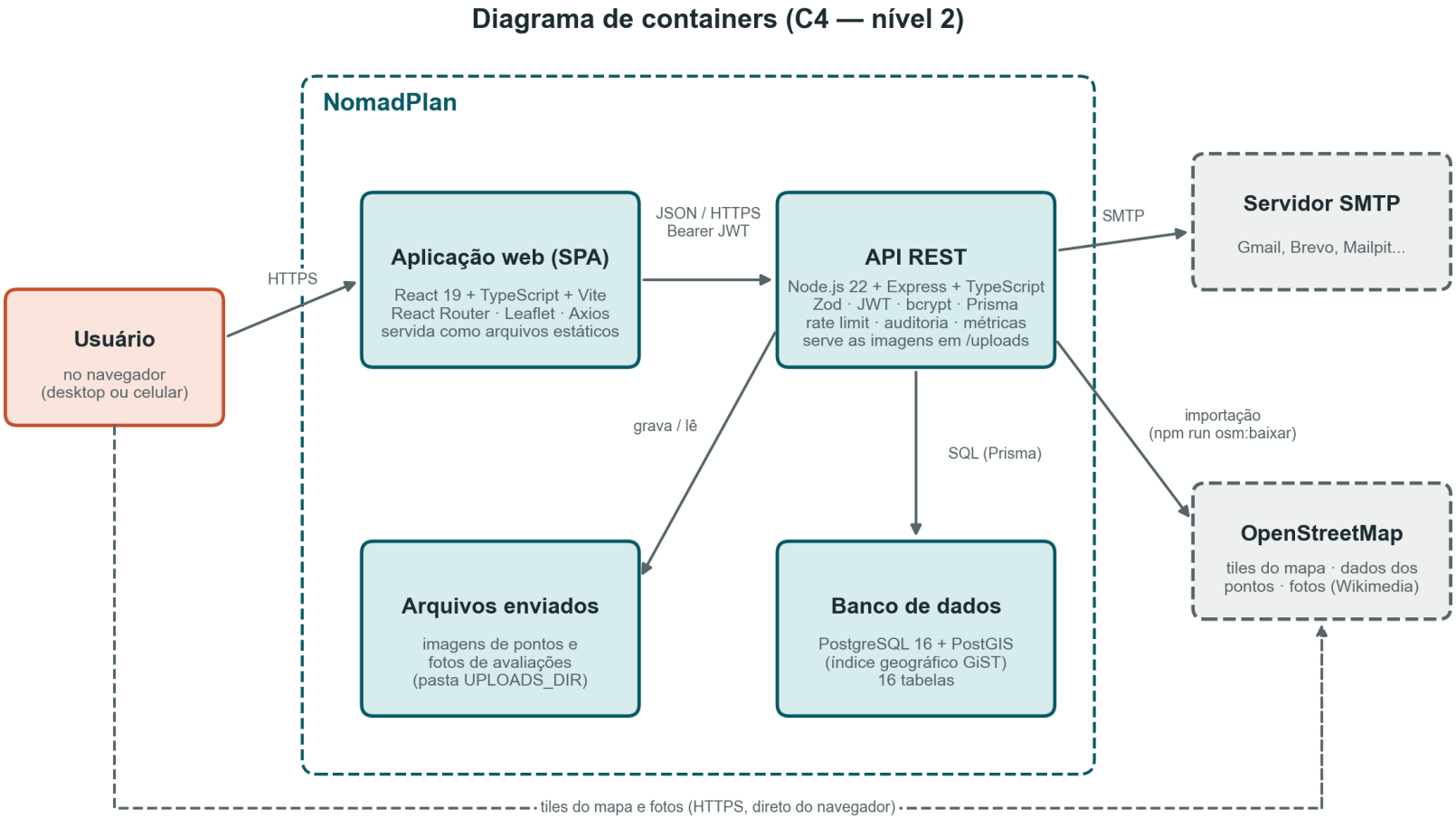



*Figura 19 — Diagrama de containers (C4, nível 2).*

**Nível 3 — Diagrama de Componentes (API REST)**

A API é organizada em três camadas. Todo pedido atravessa os **middlewares** (cabeçalhos de segurança e CORS, autenticação conferida no banco, autorização por papel, limites de taxa, acesso ao roteiro por papel de participante e validação de upload pelo conteúdo do arquivo). Em seguida, as **rotas** tratam cada área funcional. As regras de negócio ficam em **serviços** reutilizáveis (`src/lib`), independentes do Express e cobertos por testes unitários; só eles e as rotas conversam com o Prisma Client.

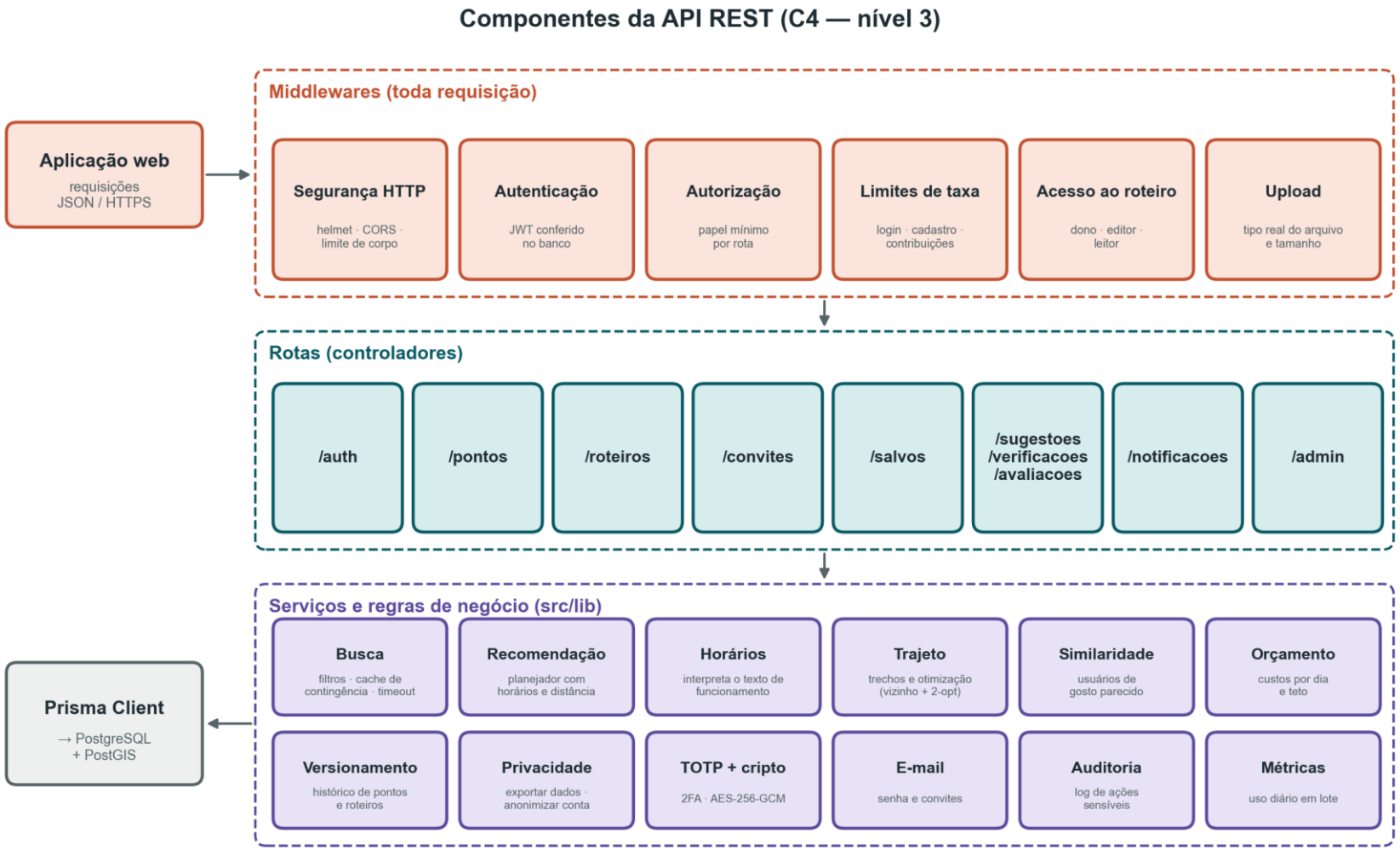



*Figura 20 — Componentes da API REST (C4, nível 3).*

### 9.2 Modelo de Dados

O banco tem **16 tabelas**. As entidades centrais são o `usuario`, o `ponto_turistico` (com uma coluna geográfica `geom`, gerada a partir de latitude e longitude e indexada por GiST, usada nas buscas por raio) e o `roteiro`, que se liga aos pontos pelos `item_roteiro` (dia, ordem, custo e observação). Em volta delas ficam as tabelas de participação social (`avaliacao`, `sugestao_edicao`, `solicitacao_verificacao`, `ponto_salvo`), de colaboração (`colaborador_roteiro`, `convite_roteiro`), de planejamento (`item_checklist`), de histórico (`versao_ponto`, `versao_roteiro`) e de governança (`notificacao`, `log_auditoria`, `metrica_diaria`). O esquema é versionado em 13 migrações SQL escritas à mão; o dicionário de dados completo está em `docs/dicionario-de-dados.md`.

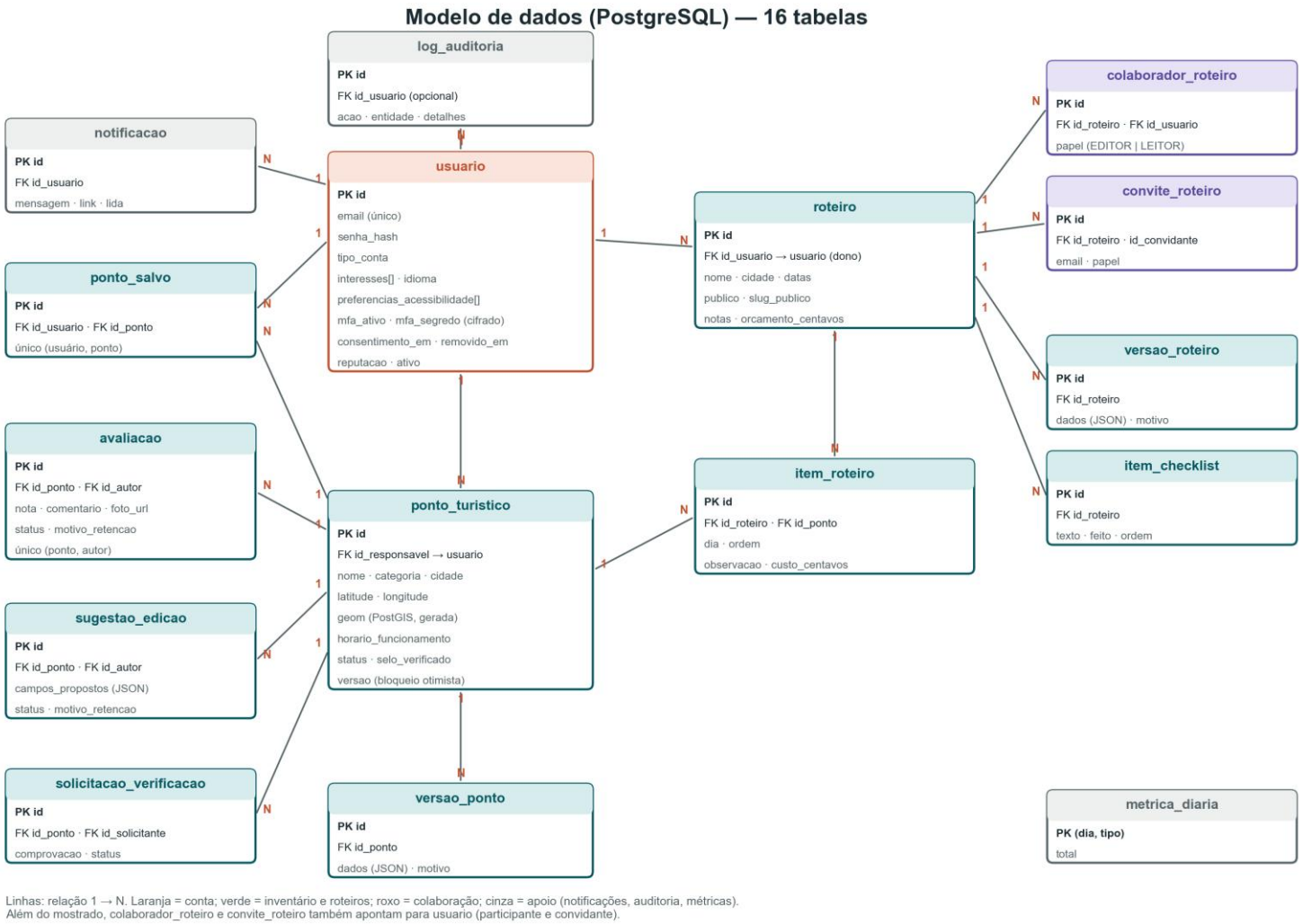



*Figura 21 — Diagrama entidade-relacionamento (DER) do NomadPlan.*

Principais decisões de modelagem:

- **Bloqueio otimista:** `ponto_turistico.versao` impede que duas edições simultâneas se sobrescrevam.
- **Avaliação única:** índice único em (usuário, ponto) garante uma avaliação por pessoa por local.
- **Histórico:** edições aprovadas e mudanças de roteiro geram versões consultáveis e restauráveis.
- **Anonimização em vez de exclusão:** a linha do usuário é mantida sem dados pessoais para preservar o histórico (seção 10.1).
- **Origem de cada ponto (RNF19):** `ponto_turistico.origem` registra se o ponto veio do cadastro de um usuário, de um arquivo importado ou da API do OpenStreetMap, e `id_osm` liga o ponto ao elemento original, o que impede duplicar numa nova importação. Com a data de criação, a de atualização, a versão e o histórico, cada registro é rastreável.

## 9.3 Principais Componentes

*Tabela 1 — Principais componentes e responsabilidades.*

| Componente | Responsabilidade |
|---|---|
| Aplicação web (SPA) | Telas em React 19 e TypeScript, rotas protegidas por papel, mapas com Leaflet, estado de sessão e de favoritos em contexto. |
| Autenticação e contas | Cadastro, login com bloqueio por tentativas, token JWT conferido no banco a cada requisição, renovação de sessão, recuperação de senha e verificação em duas etapas (TOTP, RFC 6238). |
| Pontos turísticos | Cadastro com moderação, edição com bloqueio otimista, selo de verificação, importação e exportação CSV/GeoJSON, upload de imagens validado pelo conteúdo e importação do OpenStreetMap, com fotos do Wikimedia Commons e a origem de cada ponto registrada. |
| Busca | Consulta única em SQL com filtros (categoria, preço, acessibilidade, nota, “aberto agora”, distância), ordenação, limite de tempo e resposta em cache quando a consulta demora (AF02). |
| Recomendação e roteiros | Planejador que monta o dia respeitando horários e distâncias, otimização do trajeto (vizinho mais próximo seguido de 2-opt), recomendações por similaridade entre usuários, orçamento e checklist. |
| Colaboração | Convites por e-mail, papéis de dono, editor e leitor, conferidos por middleware antes de qualquer rota do roteiro. |
| Moderação e governança | Fila de pontos, sugestões, selos e avaliações; reputação do usuário; retenção automática de conteúdo suspeito; notificações. |
| Privacidade (LGPD) | Aceite da política, exportação dos dados do usuário e anonimização da conta. |
| Auditoria e métricas | Log de ações sensíveis, alerta de possível força bruta aos administradores e métricas diárias de uso gravadas em lote. |
| Persistência | PostgreSQL 16 com PostGIS, acesso via Prisma 6 e SQL parametrizado para consultas geográficas. |

## 9.4 Stack Tecnológica

*Tabela 2 — Tecnologias utilizadas e justificativas.*

| Tecnologia | Uso | Por que foi escolhida |
|---|---|---|
| Node.js 22 + Express 4 | API REST | Ecossistema único (TypeScript) no servidor e no navegador; bom desempenho em operações de E/S; o relatório de carga (seção 11) mostrou 100 usuários simultâneos sem nenhum erro em um único processo. |
| TypeScript | Backend e frontend | Tipagem estática reduz erros de contrato entre telas e API e facilita refatorar. |
| PostgreSQL 16 + PostGIS | Banco de dados | Consultas geográficas (raio, distância) com índice espacial, integridade relacional e transações; evita servir mapas e buscas por proximidade em memória. |
| Prisma 6 | Acesso ao banco | Consultas tipadas e parametrizadas (proteção contra injeção de SQL); migrações versionadas, mantidas em SQL por causa da coluna geográfica gerada. |
| Zod | Validação de entrada | Esquemas declarativos validam tipos, tamanhos e listas em toda rota, com mensagens de erro claras. |
| JWT + bcrypt + TOTP | Autenticação | Sessão sem estado no servidor (escala por instâncias), senhas com hash lento e verificação em duas etapas aberta (RFC 6238), sem depender de serviço externo. |
| helmet, CORS, rate limit | Proteção da API | Cabeçalhos de segurança, origem restrita ao frontend e limite de tentativas por IP e por conta. |
| React 19 + Vite + React Router | Aplicação web | Componentização, carregamento sob demanda por rota e desenvolvimento rápido; SPA entregue como arquivos estáticos simples de hospedar. |
| Leaflet + OpenStreetMap | Mapas | Biblioteca leve e livre, sem chave de API nem custo por uso. |
| OpenStreetMap (Overpass API) + Wikimedia Commons | Dados dos pontos | Base aberta e gratuita que pode ser guardada no próprio banco, com atribuição (licença ODbL), ao contrário de serviços pagos como o Google Places, cujos termos não permitem isso; as fotos têm licenças livres, e o autor e a licença aparecem junto de cada uma. |
| Docker Compose | Ambiente | Sobe o banco com PostGIS em um comando e torna o ambiente reproduzível entre as máquinas da equipe. |
| node:test, Playwright e GitHub Actions | Testes e CI | 424 testes automatizados no backend (unitários e de integração, cobertura de linhas de 92%), 74 testes de ponta a ponta em navegador real e pipeline de integração contínua a cada alteração. |

## 9.5 Origem dos Dados

Os pontos turísticos da demonstração são reais: **2.025 lugares de Joinville, Florianópolis e Blumenau** vindos do **OpenStreetMap**, uma base aberta mantida por voluntários, que pode ser guardada e reutilizada desde que se dê o crédito (licença ODbL). Um script (`npm run osm:baixar`) faz uma consulta à Overpass API por cidade, escolhe os lugares de interesse turístico, como

museus, patrimônio histórico, mirantes, praias, parques, restaurantes, cafés, hotéis e shoppings, classifica cada um nas sete categorias do sistema e converte endereço, telefone, site e acessibilidade. Ficam de fora lanchonetes de rede, praças de bairro e residências. O script também busca no Wikidata e no **Wikimedia Commons** as fotos e descrições curtas, com autor e licença. O resultado fica salvo no repositório, e assim o seed e os testes não dependem de um serviço externo.

O horário de funcionamento no OpenStreetMap tem sintaxe própria (por exemplo, `Mo-Fr 09:00-18:00; Sa 09:00-13:00`) e é convertido para o formato do sistema (`Seg-Sex 9h-18h; Sáb 9h-13h`). Foram convertidos 94% dos horários encontrados; os demais usam recursos raros, como variação por mês, e ficam como não informados para não gerar um horário errado. Uma nova importação não duplica pontos e só preenche campos vazios, sem nunca sobrescrever o que a comunidade editou. Um ponto rejeitado pela moderação também não volta.

A base aberta é boa em nomes, localização e categorias, mas tem poucos horários (370 dos 2.025 pontos, 18%) e quase nenhuma informação de acessibilidade. É essa lacuna que a parte colaborativa do NomadPlan preenche: a ficha do ponto mostra a fonte dos dados e convida quem está logado a informar o que falta, e as sugestões passam pelas regras de reputação e moderação (seção 6.5). Os 370 pontos com todos os campos obrigatórios, inclusive o horário, superam a meta de cobertura mínima de 100 pontos (seção 5.3).

# 10 Segurança e Privacidade

Os controles de segurança foram organizados segundo o OWASP Top 10 (2021). O documento completo, com os riscos aceitos de forma consciente para o escopo acadêmico, está em `docs/seguranca.md`.

*Tabela 3 — Controles de segurança por categoria do OWASP Top 10.*

| Categoria OWASP | Como o NomadPlan trata |
|---|---|
| A01 Controle de acesso | Cada rota declara o papel mínimo; nos roteiros, um middleware confere se a pessoa participa e se o papel (dono, editor, leitor) basta. O token é conferido no banco a cada requisição: conta desativada ou rebaixada perde o acesso na hora. Recursos pessoais são sempre filtrados pelo usuário logado. |

| Categoria OWASP | Como o NomadPlan trata |
|---|---|
| A02 Falhas criptográficas | Senhas com bcrypt; segredo do 2FA cifrado com AES-256-GCM; JWT assinado, e a API recusa iniciar em produção com segredo curto ou de exemplo; link de redefinição de senha de uso único e 15 minutos. |
| A03 Injeção | Acesso ao banco por Prisma e SQL parametrizado; caracteres curinga das buscas escapados; entradas validadas com Zod; o React escapa o conteúdo exibido e não há HTML injetado. |
| A04 Projeto inseguro | Limites de taxa por IP e por conta; moderação obrigatória para publicar; filtro de conteúdo suspeito; bloqueio otimista contra edições simultâneas; avaliação única garantida por índice no banco. |
| A05 Configuração incorreta | helmet, CORS restrito, corpo JSON limitado, erros internos com mensagem genérica, `.env` fora do repositório e imagens servidas somente leitura. |
| A06 Componentes vulneráveis | `npm audit` sem vulnerabilidades no frontend e nos testes; no backend restam apenas alertas documentados em ferramenta de linha de comando; versões travadas por lockfile e CI com `npm ci`. |
| A07 Identificação e autenticação | Bloqueio após 3 senhas erradas por conta e 10 por IP em 15 minutos, com alerta aos administradores; 2FA com proteção contra reutilização do código; tempo de resposta do login igual para e-mails existentes e inexistentes. |
| A08 Integridade | Uploads validados pelo conteúdo (assinatura do arquivo), com limites de 5 MB (imagens) e 2 MB (importações). |
| A09 Registro e monitoramento | Log de auditoria de ações sensíveis (login, falhas, mudança de papel, importação, moderação); o e-mail digitado em tentativas falhas não é gravado; alerta de força bruta. |
| A10 SSRF | O servidor nunca busca URLs informadas por usuários; o site oficial de um ponto é apenas armazenado e exibido como link. Só o script de importação do OpenStreetMap acessa a internet, e apenas endereços fixos. |

**Riscos aceitos e recomendações para produção:** token de sessão no `localStorage` (recomenda-se cookie `HttpOnly` com proteção CSRF e Content-Security-Policy); limite de taxa em memória por instância (usar Redis com várias instâncias); TLS delegado a um proxy reverso; 2FA sem códigos de recuperação; e e-mail não verificado no cadastro. Todos estão listados com mitigação atual em `docs/seguranca.md`.

### 10.1 Privacidade e LGPD

**Dados coletados.** Nome, e-mail, cidade base, idioma, interesses, necessidades de acessibilidade e, opcionalmente, fotos enviadas em avaliações. O endereço IP de tentativas de login falhas fica no log de auditoria por segurança. A localização usada em buscas “perto de mim” não é gravada.

**Como são armazenados.** No PostgreSQL; senhas somente como hash bcrypt; o segredo do 2FA cifrado; o e-mail nunca é exibido a outros usuários. O acesso é restrito por papel e as ações administrativas são registradas em auditoria.

**Direitos do titular, exercidos pelo próprio Perfil:**

- **Consentimento:** aceite explícito da Política de Privacidade (`/privacidade`) no cadastro, registrado com data; contas antigas veem um aviso até aceitarem.
- **Portabilidade e acesso:** botão “Baixar meus dados” gera um arquivo JSON com todos os dados do usuário.
- **Exclusão:** o usuário anonimiza a conta confirmando a senha (e o código do 2FA, se ativo). Nome e e-mail são trocados por valores neutros, roteiros, favoritos, notificações e fotos são apagados, rascunhos e pontos rejeitados são removidos e o IP é apagado do log. Pontos já publicados permanecem no inventário, sem responsável.
- **Minimização:** só se coleta o necessário; o cadastro exige apenas nome, e-mail e senha.

# 11 Planejamento do Projeto

O projeto seguiu entregas incrementais por módulo, publicadas por pull requests no repositório. A tabela registra os marcos cumpridos e o que falta até a entrega final, em 05/11/2026.

*Tabela 4 — Marcos do projeto.*

| Marco | Descrição | Prazo |
|---|---|---|
| M1 | Definição do problema, requisitos e protótipo inicial; primeira versão do RFC (entrega N1) e identidade visual | Abril/2026 (até 12/04) |
| M2 | Estrutura do sistema (backend, frontend, banco com PostGIS e Docker) e roteiros básicos | 27/08 a 02/09/2026 |
| M3 | Módulos 1 a 5: autenticação e perfis, pontos com moderação, busca por proximidade e mapa, recomendação e roteiros completos, social e governança | 09 a 10/09/2026 |
| M4 | Endurecimento e identidade visual: limite de tentativas de login, correção do histórico de credenciais, redesenho da interface com a marca NomadPlan, recuperação de senha por e-mail e ajustes a partir das revisões dos colegas | 15 a 20/09/2026 |
| M5 | Conformidade com o RFC: 2FA, bloqueio de login, LGPD, validação de upload, importação, busca unificada, roteiros por dia, orçamento, checklist, versões e roteiro colaborativo; testes de integração, de ponta a ponta e de carga; CI e backup | 26 a 27/10/2026 |
| M6 | Dados reais: importação de cerca de 2 mil pontos do OpenStreetMap (Joinville, Florianópolis e Blumenau) com fotos do Wikimedia Commons, conversão de horários e registro da origem de cada ponto (RNF19) | 27 a 28/10/2026 |
| M7 | Finalização do RFC (seções 8 a 13), nomenclatura NomadPlan, revisão geral e ensaio da demonstração | até 04/11/2026 |
| Entrega | Entrega final e apresentação (02/11 é feriado) | 05/11/2026 |

**Verificação da qualidade antes da entrega.** O backend tem 424 testes automatizados (unitários e de integração, contra um banco de testes real), com cobertura de linhas de 92%; o frontend é validado por lint e build; e 74 testes de ponta a ponta rodam em navegador real. O teste de carga com 100 usuários simultâneos (26/10/2026, um único processo da API, base sintética de 5.113 pontos) não registrou nenhum erro: a latência média das buscas ficou entre 9 e 18 ms em uso normal e abaixo de 150 ms mesmo sem pausa entre as requisições, e a geração de roteiro ficou em 0,5 s de média com a base grande — dentro das metas RNF01, RNF02 e RNF04 (detalhes em `docs/relatorio-de-carga.md`).

# 12 Referências

**Documentação técnica e padrões**


- OWASP. *OWASP Top 10:2021*. https://owasp.org/Top10/
- BRASIL. *Lei nº 13.709/2018 — Lei Geral de Proteção de Dados Pessoais (LGPD)*. http://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm
- M'RAIHI, D. et al. *RFC 6238 — TOTP: Time-Based One-Time Password Algorithm*. IETF, 2011. https://datatracker.ietf.org/doc/html/rfc6238
- BROWN, S. *The C4 model for visualising software architecture*. https://c4model.com/
- NIST. *SP 800-63B — Digital Identity Guidelines: Authentication and Lifecycle Management*. https://pages.nist.gov/800-63-3/sp800-63b.html
- PostGIS. *PostGIS Documentation*. https://postgis.net/documentation/
- OpenStreetMap Foundation. *OpenStreetMap* e política de uso de tiles. https://www.openstreetmap.org/
- OpenStreetMap Foundation. *Copyright and License* (Open Database License e atribuição). https://www.openstreetmap.org/copyright
- OpenStreetMap Wiki. *Overpass API*. https://wiki.openstreetmap.org/wiki/Overpass_API
- OpenStreetMap Wiki. *Key:opening_hours* (especificação do horário de funcionamento). https://wiki.openstreetmap.org/wiki/Key:opening_hours
- Wikimedia Commons. *Reusing content outside Wikimedia*. https://commons.wikimedia.org/wiki/Commons:Reusing_content_outside_Wikimedia

### Ferramentas e bibliotecas

- Node.js, Express, TypeScript, Prisma, Zod, bcryptjs, jsonwebtoken, helmet, multer e nodemailer (backend).
- React, Vite, React Router, Axios e Leaflet (frontend).
- PostgreSQL e PostGIS; Docker e Docker Compose.
- node:test, tsx e Playwright (testes); GitHub Actions (integração contínua).

### Repositório e documentação do projeto

- Repositório: https://github.com/SandroMachiniski-SE/NomadPlan
- `README.md` — instalação e execução; `docs/api.md` — rotas da API; `docs/dicionario-de-dados.md` — modelo de dados; `docs/seguranca.md` — controles e riscos aceitos; `docs/operacao.md` — implantação, backup e recuperação; `docs/relatorio-de-carga.md` — teste de carga.

# 13 Apêndices

## Apêndice A — Documentação de apoio no repositório

*Tabela 5 — Documentos que acompanham o código.*

| Documento | Conteúdo |
|---|---|
| README.md | Visão geral, requisitos, instalação, variáveis de ambiente e como executar o sistema e os testes. |
| docs/api.md | Referência das rotas da API REST, com papéis exigidos e exemplos. |
| docs/dicionario-de-dados.md | Tabelas, colunas, relacionamentos e dados pessoais (LGPD). |
| docs/seguranca.md | Controles por categoria do OWASP, privacidade e riscos aceitos. |
| docs/operacao.md | Implantação, rollback, backup diário, restauração (RNF08), monitoramento e importação dos dados do OpenStreetMap. |
| docs/relatorio-de-carga.md | Método e resultados do teste de carga com 100 usuários. |

## Apêndice B — Contas de demonstração

O comando de seed do backend (`npm run prisma:seed`) cria contas de demonstração para cada perfil — administrador, moderador, negócio e morador, todas com o domínio `@nomadplan.local` — e carrega os 2.025 pontos reais de Joinville, Florianópolis e Blumenau baixados do OpenStreetMap (arquivos em `backend/prisma/dados/osm/`). O seed recusa rodar em produção; lá, os pontos entram com `npm run osm:importar`. Para atualizar os dados ou incluir

outra cidade: `npm run osm:baixar -- <cidade>` seguido de `npm run osm:importar`. As senhas estão no README do repositório.

### Apêndice C — Como executar e testar

- Banco: `docker compose up -d postgres`.
- Backend: `cd backend && npm ci && npx prisma migrate deploy && npm run prisma:seed && npm run dev` (porta 3333).
- Frontend: `cd frontend && npm ci && npm run dev` (porta 5173).
- Testes do backend: `npm run test:cobertura` (unitários e de integração, com limite mínimo de 60% de cobertura de linhas).
- Testes de ponta a ponta: `cd e2e && npm ci && npx playwright test`.
- Teste de carga: `node e2e/carga/carga.mjs` (instruções em `docs/relatorio-de-carga.md`).

# 14 Parecer do Comitê de Avaliação

**(A ser preenchido pelos professores)**

**Avaliador 1:** _____

**Status:** [ ] Aprovado [ ] Ajustar

**Observações:**

**Avaliador 2:** _____

**Status:** [ ] Aprovado [ ] Ajustar

**Observações:**

**Avaliador 3:** _____

**Status:** [ ] Aprovado [ ] Ajustar

**Observações:**

**Participação no projeto de Diego Cunha:**


DC811cunha / metrologia-software
Type / to search
Code
Issues 1
Pull requests
Actions
Projects
Security and quality
Insights
Comentário sobre o SoftMeter. #1
Edit
New issue
Open
SandroMachiniski-SE opened now
O SoftMeter apresenta uma proposta inovadora ao trazer a lógica da metrologia industrial para o software. Um ponto que pode enriquecer ainda mais o projeto é explorar como o Laudo de Conformidade poderia ser aplicado em metodologias ágeis, funcionando como um artefato de validação contínua em cada sprint. Isso ampliaria a utilidade da solução tanto em ambientes acadêmicos quanto em contextos de mercado.
Assignees
No one assigned
Labels
No labels
Projects
No projects
Milestone